Contents

1.Write Java program to print “Hello World”. 5

2. Write a program in Java to read a number and tests if it is even or odd. 6

3. Write a program that finds the largest element in an array. 6

4. Write a program in java that finds second largest element in an array. 7

5. Given three numbers, write a Java program to read three numbers from keyword and print out the largest of them. 8

6. Write a Java program reads a character and check if it is alphabet or not. 9

7. Write a program that checks if the array is sorted or not. 10

8. Write a Java program to read two integer values m and n and to decide whether m is a multiple of n. 11

9. Write a Java program that will obtain the length and width of a rectangle from the user and compute its area and perimeter. 12

10. Write a Java program that reads radius of circle and finds area and circumference. 13

11. Write a Java program that reads two numbers and swap them. 14

12. Write a Java program that reads P,T, and R and find simple interest and amount. 15

13. Write a Java program that sorts a given array. 16

14. Write a Java program that finds factorial of a positive number using recursive method... 17

15. Write a Java program that finds nth Fibonacci number using recursive method. 18

16. Write a Java program that find product of two numbers using recursive method. 19

17. Write a Java program that find x to the power y using recursive method 20

18. Write Java program to print prime numbers from 300 to 500 using method. 21

19. Write a Java program to find the largest number among four different numbers using conditional operator. 22

20.Write a java program that swaps two elements in an array. 22

21. Write a java program that prints elements of an array using for each type of loop. 23

22.A non-empty array A of length n is called on array of all possibilities if it contains all numbers between 0 and A.length-1 inclusive. Write a method named is All Possibilities that accepts an integer array and returns 1 if the array is an array of all possibilities, otherwise it returns 0. 24

23.Write a Java program that finds sum of two and three numbers using concept of method overloading. 25

24.Write a Java program that finds areas of different geometric shapes using concept of method overloading. 26

25.Create a class Number with three int instance variable x , y and z. The class will have one constructor. The class also will contain member function getMax () that will return the largest number. Create a main method that will create an object of Number and will print the largest number.....26

26.Write a Java program to add two complex numbers.....27

27.Write a Java program to add two Time(hr,min,sec) objects.....28

28. Create a class Swapper class with two integer instance variable x and y and constructor with two parameters that initializes the two variables. Also include three member functions: A getX () that returns x, a getY () function that returns y, a void swap () method that swaps the values of x and y. Then define a main() method to create an object of Swapper class and swap the value of instance variables. 29

29. Create a class Date with three integer instance variables named day, month, year. It has a constructor with three parameters for initializing the instance variables, and it has one-member function named daySinceJan1 (). It computes and returns the number of days since January 1 of the same year, including January 1 and the day in the Date object. For example, if day is a Date object with day = 1, month = 3 and year = 2000, then the call date.daySinceJan1() should return 61 since there are 61 days between the dates of January 1, 2000, and March 1, 2000, including January 1 and March 1. Then define main () method to handle Date class. Don't forget leap years.....30

30.Create a USMoney class with two integer instance variables dollars and cents. Add a constructor with two parameters for initializing a USMoney object. The constructor should check that the cent value is between 0 and 99 and, if not, transfer some cents to the dollars variables to make it between 0 and 99. For example, if x is a USMoney object with 5 dollars and 80 cents, and if y is a USMoney object with 1 dollar and 90 cents, then x.plus (y) will return a new USMoney object with 7 dollars and 70 cents. Also, create a main () method that creates to objects of USMoney class and add them. 31

Output: 32

31.Create a Person class with private instance variables for person's name and birth date. Add appropriate functions for these variables. Then create a subclass CollegeGraduate with private instance variables for the student's GPA and year of graduation and appropriate functions for these variables. Don't forget to include appropriate constructor constructors for your classes. Then define main () method that demonstrates your classes.....32

32.Create a class Box with fields width, height and depth. Add methods getArea () and getVolume (). Use suitable constructors. From main () method create an object of Box class and find its area as volume.....34

33. Create a class Room with instance variables length and breadth. Add one function getArea () that returns the area of the room. Create a subclass MyRoom and add one instance

variable height. Add one function getVolume () that returns the volume. Then define main () method that creates two MyRoom objects and find area and volumes of both rooms. 35

34.Create a class Box with instance variables length, breadth and height. Add one method getVolume () to compute the volume of box. Use suitable constructors. Create a subclass BoxWeight that extends Box that add one variable weight. Add one function getWeight () that displays the weight of box to this class. Add suitable constructors. Create one more subclass class Shipment that extends BoxWeight. Add one function getCost () that displays the cost of the box. Add suitable constructors. Then define main () method that creates an object of Shipment that initializes the instance variables through constructor. 36

35.Create an abstract class Figure with two instance variables dim1 and dim2. Add suitable constructors. Add one abstract function called getArea (). Create two subclass called Rectangle and Triangle. Add function getArea () to both of the classes that will find the area of respective figures. Then define main () method that creates an object of each classes and find the area of triangle and rectangle. 37

36. Write a Java program to demonstrate divide by zero exception. 38

37.Write a Java program to demonstrate array index bounds exception. 39

38.Write a Java Program to demonstrate null exception. 40

39. Write a Java program to demonstrate multiple catch statements. 41

40. Write a Java program to demonstrate custom exception 41

42.Write a Java program to compare two strings. 43

43.Write a Java program to compare some portion of two Strings. 44

44. Write a Java program that replaces some characters in Strings. 45

45. Write a Java program to reads contents of a file using character stream. 46

46. Write a Java program to write some lines of text to file using character stream. 47

47.Write a Java program that reads contents of same file using character stream. 48

48.Write a Java program that writes line of text to file using byte stream. 49

49. Write a Java program that reads contents of file using byte stream. 50

50.Write a Java program to write objects to file. 51

51. Write a Java program to read objects from file. 52

52.Write a Java program to demonstrate random access process 53

53.Write a Java program to print 10 random numbers between 1 to 100. 54

54. Write a Java program that creates a class called Stack and then implements push() and pop() operations. 54

55.Create an interface Exam with methods setExam(String division, int mark) and showExam(), create a class named test that implements the interface Exam and then display the records. 56

56. Write a Java program to create a class Mobile (type, phone_no). Customize the exception such that if the user give phone_no having less than or greater than 10 digit, then the program has to throw an exception with message “Invalid Phone Number”. 57

57. Create a class named Movie (id, genre). Write the object of Movie class into file named “Comedy.dat” having comedy as genre. 58

58.Write a program to create a class student with data member roll and name. sort the 10 objects of this class on the basis of name. 60

59.Create a class named Book with instance variables tile and price. Add a method named setVar to pass parameters for title and price. Add another method named showVar to display values of these variables. Now in main(), declare 4 objects of book and display the records of book that starts with “Java”. 61

60.Create a Shape interface having methods area() and perimeter(). Create two subclasses, Circle and Rectangle that implements the Shape interface. Create a class Sample with main method and demonstrate the area and perimeters of both the Shape classes. You need to handle the values of length, breadth and radius in respective classes to calculate their area and perimeter. 62

61. Write a Java program to show use of Wrapper classes. 63

62.Write a Java program to illustrate inner class. 64

63.Write a Java program that shows the use of generic class. 65

64.Write a Java program to show generic method. 66

65. Create a class Student with private member variables name and percentage. Write methods to set, display and return values of private variables in the Student class. Create 10 different objects of the student class, set the values, ad display name of Student who have highest average_marks in the main method of another class named StudentDemo. 66

66.Write a program to illustrate the concept of ArrayIndexOutOfBoundsException. 67

67.Write a Java program to read data file form the file “text.txt” and write the data file into the file “best.txt”. 68

68.Write a Java program writes line of text to existing file. Also read the content of this file and write down on monitor. 69

69. Write a Java program reads N names of students and then sort them in ascending order. 70

70.Write a Simple GUI program that displays “hello World” in a text field. The program should display if user clicks a button. 71

71. Write GUI program using Swing components to find sum and difference of two numbers. Use two text fields for giving input and a label for output. The program should display sum if user presses mouse and difference if user release mouse. 72

1.Write Java program to print "Hello World".

Java Programming Language

Definition:

Java is a high-level, object-oriented, platform-independent programming language developed by Sun Microsystems (now Oracle). It is designed to be simple, secure, and portable, allowing programs to run on any device that has a Java Virtual Machine (JVM).

Key Features:

1. **Object-Oriented:** Java supports classes, objects, inheritance, polymorphism, encapsulation, and abstraction.
2. **Platform-Independent:** Java programs are compiled into bytecode, which can run on any platform with a JVM ("Write Once, Run Anywhere").
3. **Robust & Secure:** Java provides strong memory management, exception handling, and security features.
4. **Multithreaded:** Java supports concurrent execution of multiple threads.
5. **Automatic Memory Management:** Java has garbage collection to manage memory automatically.

Key Components:

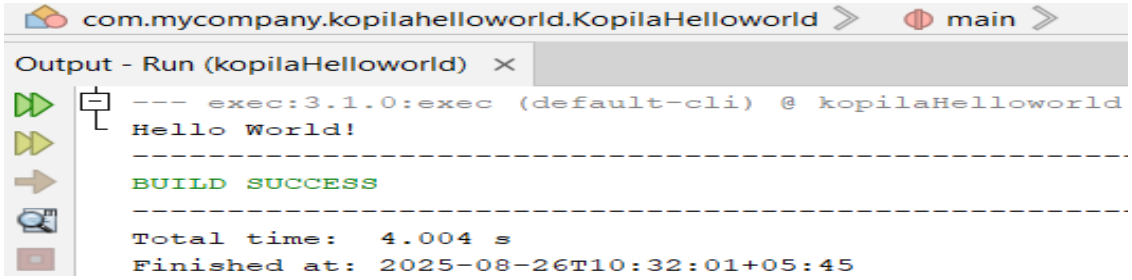
JVM: Interprets bytecode into machine code for the host system.

JRE (Java Runtime Environment): Contains JVM and libraries to run Java programs.

JDK (Java Development Kit): Contains JRE, compiler, and tools for development.

Source code:

```
package com.mycompany.kopilahelloworld;

public class KopilaHelloworld {

public static void main(String[] args) {

  System.out.println("Hello World!");

}}
```

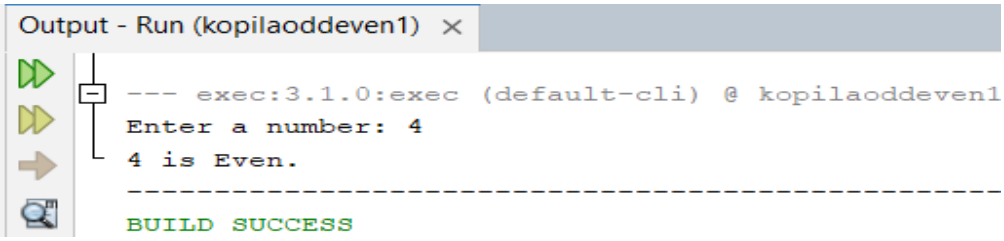


2. Write a program in Java to read a number and tests if it is even or odd.

Source code:

```
package com.mycompany.kopilaoddeven1;

import java.util.Scanner;

public class Kopilaoddeven1 {

   public static void main(String[] args) {

         try (Scanner sc = new Scanner(System.in)) {

               System.out.print("Enter a number: ");

         int number = sc.nextInt();

            if (number % 2 == 0) {

            System.out.println(number + " is Even.");

         } else {

            System.out.println(number + " is Odd.");

         }}

   }}
```

Output:



3. Write a program that finds the largest element in an array.

Introduction:

In Java, an **array** is a collection of elements of the same type stored at contiguous memory locations.
To find the **largest element** in an array, we compare each element with the current maximum and update it if a larger value is found.

Syntax of an Array in Java:

datatype[] arrayName = new datatype[size];

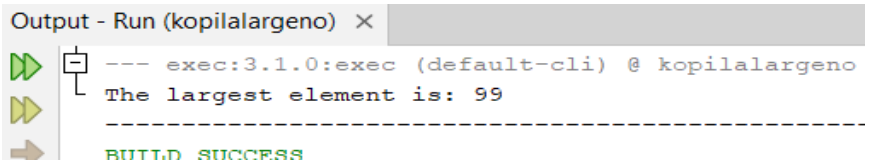
OR, datatype[] arrayName = {value1, value2, value3, ...};

Source code:

```
package com.mycompany.kopilalargeno;
```

```
public class Kopilalargeno {
    public static void main(String[] args) {
        int[] arr = {10, 25, 5, 99, 42};
        int largest = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] > largest) {
                largest = arr[i];
            }}
        System.out.println("The largest element is: " + largest);
    }}
```

Output:



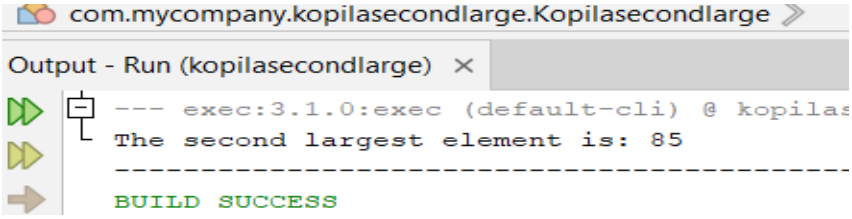
4. Write a program in java that finds second largest element in an array.

Source Code:

```
package com.mycompany.kopilasecondlarge;
public class Kopilasecondlarge {
    public static void main(String[] args) {
        int[] arr = {10, 25, 5, 99, 42, 85};
        int largest = Integer.MIN_VALUE;
        int secondLargest = Integer.MIN_VALUE;
        for (int num : arr) {
            if (num > largest) {
                secondLargest = largest;
                largest = num;
            } else if (num > secondLargest && num < largest) {
                secondLargest = num;
            }}
        if (secondLargest == Integer.MIN_VALUE) {
```

```
        System.out.println("No second largest element.");
    } else {
        System.out.println("The second largest element is: " + secondLargest);
    }}}
```

Output:



5. Given three numbers, write a Java program to read three numbers from keyword and print out the largest of them.

Introduction: In Java, we can take input from the keyboard using the Scanner class. To find the largest among three numbers, we compare them using **if-else statements** or the **Math.max()** method.

Syntax:

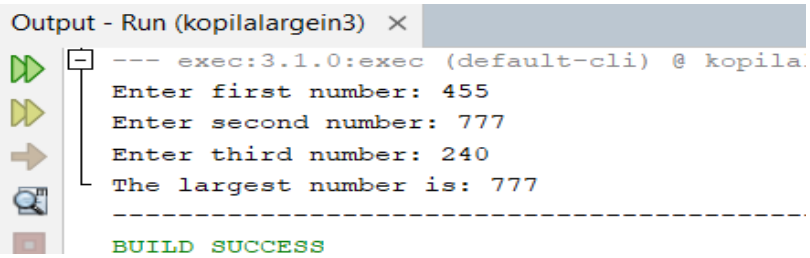
```
import java.util.Scanner;   // for input
Scanner sc = new Scanner(System.in);
int a = sc.nextInt();
```

Source code:

```
package com.mycompany.kopilalargein3;
import java.util.Scanner;
public class Kopilalargein3  {
   public static void main(String[] args) {
       Scanner sc = new Scanner(System.in);
       System.out.print("Enter first number: ");
       int a = sc.nextInt();
    System.out.print("Enter second number: ");
       int b = sc.nextInt();
       System.out.print("Enter third number: ");
       int c = sc.nextInt();
       int largest;
```

```
        if (a >= b && a >= c) {
            largest = a;
        } else if (b >= a && b >= c) {
            largest = b;
        } else {
            largest = c; }
        System.out.println("The largest number is: " + largest);
        sc.close();
    }}
```

Output:



6. Write a Java program reads a character and check if it is alphabet or not.

Source code:

```
package com.mycompany.kopilacheckalphabet;
import java.util.Scanner;
public class Kopilacheckalphabet {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a character: ");
        char ch = sc.next().charAt(0);
        if ((ch >= 'a' && ch <= 'z') || (ch >= 'A' && ch <= 'Z')) {
            System.out.println(ch + " is an Alphabet.");
        } else {
            System.out.println(ch + " is NOT an Alphabet.");   }
        sc.close();    }}
```

Output:

```
com.mycompany.kopilacheckalphabet
Output - Run (kopilacheckalphabet)
Enter a character: p
p is an Alphabet.
------------------------
BUILD SUCCESS
```

```
com.mycompany.kopilacheckalphabet.
put - Run (kopilacheckalphabet)
Enter a character: 1
1 is NOT an Alphabet.
------------------------
BUILD SUCCESS
```

7. Write a program that checks if the array is sorted or not.

Introduction: In Java, an **array** is a collection of elements of the same type. To check if an array is **sorted in ascending order**, we compare each element with its next element.

- If every element is less than or equal to the next one, the array is sorted.
- Otherwise, it is **not sorted**.

Syntax:

```
int[] arr = {values};      // define array
boolean sorted = true;     // assume array is sorted
for (int i = 0; i < arr.length - 1; i++) {
   if (arr[i] > arr[i + 1]) {
      sorted = false;    // found unsorted pair
      break; }}
if (sorted) {
   // array is sorted
} else {
   // array is not sorted
}
```

Source Code:

```
package com.mycompany.kopilacheckarraysort;
public class Kopilacheckarraysort  {
   public static void main(String[] args) {
      int[] arr = {10, 20, 30, 40, 50}; // example array
      boolean sorted = true;
      for (int i = 0; i < arr.length - 1; i++) {
         if (arr[i] > arr[i + 1]) {
            sorted = false;
```

```
            break;      }}
    if (sorted) {
        System.out.println("The array is sorted in ascending order.");
    } else {
        System.out.println("The array is not sorted.");
    }}}
```

Output:

```
ut - Run (kopilacheckarraysort)
--- exec:3.1.0:exec (default-cli) @ kopila
The array is sorted in ascending order.
------------------------------------------
BUILD SUCCESS
```

8. Write a Java program to read two integer values m and n and to decide whether m is a multiple of n.

Introduction: In Java, to check if a number m is a **multiple of** another number n, we use the **modulus operator %**.

- If m % n == 0, then m is a multiple of n.
- Otherwise, it is **not a multiple**.

Source code:

```
package com.mycompany.kopilacheckmultiple;
import java.util.Scanner;
public class Kopilacheckmultiple {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number (m): ");
        int m = sc.nextInt();
        System.out.print("Enter second number (n): ");
        int n = sc.nextInt();
        if (n == 0) {
            System.out.println("Division by zero is not allowed.");
        } else if (m % n == 0) {
            System.out.println(m + " is a multiple of " + n);
```

```
        } else {
            System.out.println(m + " is NOT a multiple of " + n);    }
        sc.close();
    }}
```

Output:

```
com.mycompany.kopilacheckmultiple.Kopilacheckmultiple
ut - Run (kopilacheckmultiple)
--- exec:3.1.0:exec (default-cli) @ kopila
Enter first number (m): 55
Enter second number (n): 11
55 is a multiple of 11
------------------------------------------
BUILD SUCCESS
```

9. Write a Java program that will obtain the length and width of a rectangle from the user and compute its area and perimeter.

Introduction: In Java, we can compute the **area** and **perimeter** of a rectangle if we know its **length** and **width**:

- **Area** = length × width
- **Perimeter** = 2 × (length + width)

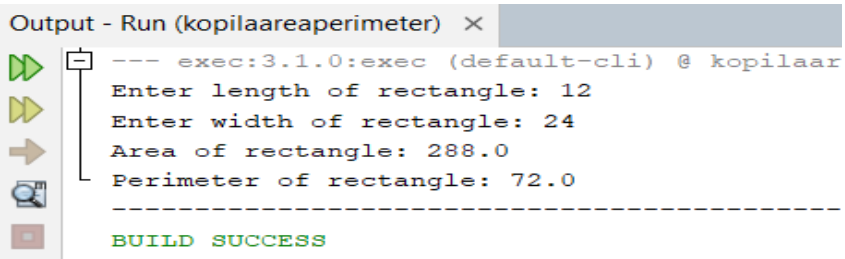
We can read the length and width from the user using the Scanner class.

Source code:

```
package com.mycompany.kopilaareaperimeter;
import java.util.Scanner;
public class Kopilaareaperimeter {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter length of rectangle: ");
        double length = sc.nextDouble();
        System.out.print("Enter width of rectangle: ");
        double width = sc.nextDouble();
        double area = length * width;
        double perimeter = 2 * (length + width);
        System.out.println("Area of rectangle: " + area);
```

```
        System.out.println("Perimeter of rectangle: " + perimeter);
        sc.close();
    }}
```

Output:



10. Write a Java program that reads radius of circle and finds area and circumference.

Introduction: In Java, we can calculate the **area** and **circumference** of a circle if we know its **radius**:

- **Area** = π × radius²
- **Circumference** = 2 × π × radius

We can use Math.PI for the value of π and read the radius from the user using the Scanner class.

Source Code:

```
package com.mycompany.kopilacircle;
 import java.util.Scanner;
public class Kopilacircle {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter radius of circle: ");
        double radius = sc.nextDouble();
        double area = Math.PI * radius * radius;
        double circumference = 2 * Math.PI * radius;
        System.out.println("Area of circle: " + area);
        System.out.println("Circumference of circle: " + circumference);
        sc.close();    }}
```

Output:

```
Output - Run (kopilacircle)
--- exec:3.1.0:exec (default-cli) @ kopilacircl
Enter radius of circle: 7
Area of circle: 153.93804002589985
Circumference of circle: 43.982297150257104
------------------------------------------------
BUILD SUCCESS
```

11. Write a Java program that reads two numbers and swap them.

Introduction: In Java, **swapping two numbers** means exchanging their values. We can do this using a **temporary variable** or **arithmetic operations**.

- Using a **temporary variable** is the simplest and safest method.

Source Code:

```
package com.mycompany.kopilaswap;

import java.util.Scanner;

public class Kopilaswap{

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter first number: ");

      int a = sc.nextInt();

      System.out.print("Enter second number: ");

      int b = sc.nextInt();

      int temp = a;

      a = b;

      b = temp;

      System.out.println("After swapping: ");

      System.out.println("First number: " + a);

      System.out.println("Second number: " + b);

      sc.close(); }}
```

```
ut - Run (kopilaswap)
--- exec:3.1.0:exec (default-cli) @ kopila
Enter first number: 23
Enter second number: 44
After swapping:
First number: 44
Second number: 23
------------------------------------------
BUILD SUCCESS
```

12. Write a Java program that reads P,T, and R and find simple interest and amount.

Introduction: In Java, **Simple Interest (SI)** is calculated using the formula:

SI=P×R×T /100 , where:

- **P** = Principal amount
- **R** = Rate of interest (per year)
- **T** = Time (in years)

The **Total Amount (A)** is calculated as: A=P+SI

Source Code:

```
package com.mycompany.kopilainterest;

import java.util.Scanner;

public class Kopilainterest {

   public static void main(String[] args) {

      Scanner sc = new Scanner(System.in);

      System.out.print("Enter Principal (P): ");

      double P = sc.nextDouble();

      System.out.print("Enter Rate of Interest (R in %): ");

      double R = sc.nextDouble();

      System.out.print("Enter Time (T in years): ");

      double T = sc.nextDouble();

      double SI = (P * R * T) / 100;

      double Amount = P + SI;

     System.out.println("Simple Interest: " + SI);

      System.out.println("Total Amount: " + Amount);

      sc.close();    }}
```

```
com.mycompany.kopilainterest.Kopilainterest >  main >
ut - Run (kopilainterest)  ×
--- exec:3.1.0:exec (default-cli) @ kopilain
Enter Principal (P): 10000
Enter Rate of Interest (R in %): 8
Enter Time (T in years): 5
Simple Interest: 4000.0
Total Amount: 14000.0
------------------------------------------
BUILD SUCCESS
```

13. Write a Java program that sorts a given array.

Introduction: In Java, **sorting an array** means arranging its elements in **ascending order** (smallest to largest) or **descending order**.

- The simplest way is to use **nested loops** and compare elements (Bubble Sort).
- Alternatively, we can use **Arrays.sort()** for a built-in method.

Syntax:

```
for (int i = 0; i < arr.length - 1; i++) {
   for (int j = 0; j < arr.length - 1 - i; j++) {
      if (arr[j] > arr[j + 1]) {
         int temp = arr[j];
         arr[j] = arr[j + 1];
         arr[j + 1] = temp;
      }
 }}
```

Source Code:

```
package com.mycompany.kopilasortarray;
import java.util.Arrays;
public class Kopilasortarray {
   public static void main(String[] args) {
      int[] arr = {25, 10, 5, 42, 18};
      Arrays.sort(arr);  // sort in ascending order
      System.out.println("Sorted array: ");
      for (int num : arr) {
         System.out.print(num + " ");
      }}}
```

Output:

```
ut - Run (kopilasortarray)
--- exec:3.1.0:exec (default-cli) @ kopilaso
Sorted array:
5 10 18 25 42
------------------------------------------
BUILD SUCCESS
```

14. Write a Java program that finds factorial of a positive number using recursive method.

Introduction: In Java, the **factorial** of a positive integer n is the product of all integers from 1 to n: $n! = n \times (n-1) \times (n-2) \cdots \times 1$

A **recursive method** calls itself to compute the factorial:

- Base case: factorial(0) = 1
- Recursive case: factorial(n) = n * factorial(n-1)

Syntax:

```
int factorial(int n) {
   if (n == 0) return 1;      // base case
   else return n * factorial(n - 1); // recursive call
}
```

Code:

```
package com.mycompany.kopilafactorial;
import java.util.Scanner;
public class Kopilafactorial {
   public static int factorial(int n) {
      if (n == 0)
         return 1;
      else
         return n * factorial(n - 1);   }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter a positive number: ");
      int num = sc.nextInt();
      if (num < 0) {
         System.out.println("Factorial is not defined for negative numbers.");
      } else {
         int result = factorial(num);
         System.out.println("Factorial of " + num + " is: " + result);     }
      sc.close();    }}
```

Output:

```
- Run (kopilafactorial)
Enter a positive number: 5
Factorial of 5 is: 120
----------------------------
BUILD SUCCESS
```

15. Write a Java program that finds nth Fibonacci number using recursive method.

Introduction : The **Fibonacci series** is a sequence of numbers where each number is the sum of the two preceding ones: 0,1,1,2,3,5,8,13,…

In Java, we can find the **nth Fibonacci number** using a **recursive method**:

- Base cases: fib(0) = 0 , fib(1) = 1
- Recursive case: fib(n) = fib(n-1) + fib(n-2)

Syntax:

```
int fib(int n) {
   if (n == 0) return 0;
   if (n == 1) return 1;
   return fib(n-1) + fib(n-2); // recursive call
}
```

Code:

```
package com.mycompany.kopilafibonacci;
import java.util.Scanner;
public class Kopilafibonacci {
   public static int fib(int n) {
      if (n == 0) return 0;
      if (n == 1) return 1;
      return fib(n - 1) + fib(n - 2);    }
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter n (position in Fibonacci series): ");
      int n = sc.nextInt();
      if (n < 0) {
         System.out.println("Please enter a non-negative number.");
```

```
    } else {
        int result = fib(n);
        System.out.println("The " + n + "th Fibonacci number is: " + result); }
    sc.close();
  }}
```

Output:

```
: - Run (kopilafibonacci)
] --- exec:3.1.0:exec (default-cli) @ kopilafibo
  Enter n (position in Fibonacci series): 6
  The 6th Fibonacci number is: 8
  ------------------------------------------------
  BUILD SUCCESS
```

16. Write a Java program that find product of two numbers using recursive method.

Introduction: In Java, we can find the **product of two numbers** using **recursion**.

- The idea is that multiplication can be expressed as repeated addition:

  product(a,b) = a + product(a,b−1)

- Base case: product(a, 0) = 0
- Recursive case: product(a, b) = a + product(a, b-1)

Syntax:

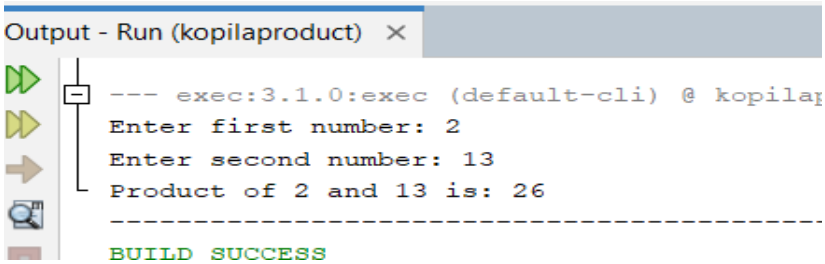
```
int product(int a, int b) {
  if (b == 0) return 0;       // base case
  return a + product(a, b - 1); // recursive call
}
```

Code:

```
package com.mycompany.kopilaproduct;
import java.util.Scanner;
public class Kopilaproduct {
  public static int product(int a, int b) {
    if (b == 0) return 0;
    return a + product(a, b - 1);    }
  public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter first number: ");
int a = sc.nextInt();
System.out.print("Enter second number: ");
int b = sc.nextInt();
if (a < 0 || b < 0) {
    System.out.println("Please enter non-negative numbers.");
} else {
    int result = product(a, b);
    System.out.println("Product of " + a + " and " + b + " is: " + result);    }
sc.close();    }}
```

Output:



17. Write a Java program that find x to the power y using recursive method

Introduction : In Java, we can calculate **x to the power y** (x^y) using **recursion**.

- The idea is: x^y = x×x^(y−1)
- Base case: x^0 = 1
- Recursive case: x^y = x * power(x, y-1)

Code:

```
package com.mycompany.kopilarecursionpower;
import java.util.Scanner;
public class Kopilarecursionpower {
    public static int power(int x, int y) {
        if (y == 0) return 1;
        return x * power(x, y - 1);   }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
System.out.print("Enter base (x): ");
int x = sc.nextInt();
System.out.print("Enter exponent (y): ");
int y = sc.nextInt();
if (y < 0) {
    System.out.println("Please enter a non-negative exponent.");
} else {
    int result = power(x, y);
    System.out.println(x + " to the power " + y + " is: " + result);    }
sc.close();   }}
```

Output:

```
- Run (kopilarecursionpower)
--- exec:3.1.0:exec (default-cli) @ kopilarecursion
Enter base (x): 2
Enter exponent (y): 5
2 to the power 5 is: 32
------------------------------------------------------------
BUILD SUCCESS
```

18. Write Java program to print prime numbers from 300 to 500 using method.

Code:

```
package com.mycompany.kopilaprime;
   public class Kopilaprime {
   public static boolean isPrime(int n) {
      if (n <= 1) return false;
      for (int i = 2; i <= Math.sqrt(n); i++) {
         if (n % i == 0) return false;  }
      return true;    }
   public static void main(String[] args) {
      System.out.println("Prime numbers from 300 to 500:");
      for (int i = 300; i <= 500; i++) {
         if (isPrime(i)) {
            System.out.print(i + " ");
         }} }}
```

Output:

```
- Run (kopilaprime)
--- exec:3.1.0:exec (default-cli) @ kopilaprime ---
Prime numbers from 300 to 500:
307 311 313 317 331 337 347 349 353 359 367 373 379 383 389 397 401 409 419 421 431 433 439 443 449 457 461 463 467 479 487 491 499
------------------------------------------------------------------------
BUILD SUCCESS
```

19. Write a Java program to find the largest number among four different numbers using conditional operator.

Introduction: In Java, the conditional operator (?:) can be used as a shorthand for if-else statements. To find the largest number among four numbers, we can compare them step by step using the conditional operator:
largest=(a>b?a:b)>(c>d?c:d)?(a>b?a:b):(c>d?c:d);

Code:

```
package com.mycompany.kopilaconditionallargest;
import java.util.Scanner;
public class Kopilaconditionallargest {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first number: ");
      int a = sc.nextInt();
      System.out.print("Enter second number: ");
      int b = sc.nextInt();
      System.out.print("Enter third number: ");
      int c = sc.nextInt();
      System.out.print("Enter fourth number: ");
      int d = sc.nextInt();
      int largest = (a > b ? a : b) > (c > d ? c : d) ? (a > b ? a : b) : (c > d ? c : d);
     System.out.println("The largest number is: " + largest);
      sc.close();    }}
```

Output:

```
: - Run (kopilaconditionallargest)  ×

| --- exec:3.1.0:exec (default-cli) @ kopilaconditionallargest
  Enter first number: 21
  Enter second number: 55
  Enter third number: 23
  Enter fourth number: 12
  The largest number is: 55
  ------------------------------------------------------------
  BUILD SUCCESS
```

20.Write a java program that swaps two elements in an array.

Introduction: In Java, swapping two elements in an array means exchanging the values at two specified indices.
We usually use a temporary variable to hold one value while swapping.
The basic idea: temp = arr[i];
arr[i] = arr[j];
arr[j] = temp;

Code:

```
package com.mycompany.kopilaswaparray;
import java.util.Scanner;
public class Kopilaswaparray {
      public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int[] arr = {10, 20, 30, 40, 50};
      System.out.println("Original array:");
      for (int num : arr) {
         System.out.print(num + " ");    }
```

```
System.out.println();
System.out.print("Enter index of first element to swap: ");
int i = sc.nextInt();
System.out.print("Enter index of second element to swap: ");
int j = sc.nextInt();
if (i >= 0 && i < arr.length && j >= 0 && j < arr.length) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
    System.out.println("Array after swapping:");
    for (int num : arr) {
        System.out.print(num + " ");
    }
} else {
    System.out.println("Invalid indices.");    }
sc.close();   }}
```

Output:

```
- Run (kopilaswaparray)
--- exec:3.1.0:exec (default-cli) @ kopilaswaparray
Original array:
10 20 30 40 50
Enter index of first element to swap: 2
Enter index of second element to swap: 4
Array after swapping:
10 20 50 40 30
------------------------------------------------------
BUILD SUCCESS
```

21. Write a java program that prints elements of an array using for each type of loop.

Introduction: In Java, a for-each loop (also called enhanced for loop) is used to iterate over arrays or collections without using an index.

- **Syntax:** for (datatype element : array)
- It automatically accesses each element of the array in sequence.

Code:

```
package com.mycompany.kopilaforeach;

public class Kopilaforeach {

   public static void main(String[] args) {

      int[] arr = {10, 20, 30, 40, 50};

      System.out.println("Array elements are:");

      for (int num : arr) {

         System.out.print(num + " ");   } }}
```

Output:

```
ut - Run (kopilaforeach)
--- exec:3.1.0:exec (default-cli) @ kopilaforeach
Array elements are:
10 20 30 40 50
------------------------------------------------------
BUILD SUCCESS
```

22.A non-empty array A of length n is called on array of all possibilities if it contains all numbers between 0 and A.length-1 inclusive. Write a method named is All Possibilities that accepts an integer array and returns 1 if the array is an array of all possibilities, otherwise it returns 0.

Introduction : An array is called an array of all possibilities if it contains all numbers from 0 to n-1 (where n is the length of the array).

- Example: {1, 2, 0, 3} is valid for length 4 because it has {0,1,2,3}.
- Example: {0, 1, 2, 4} is not valid for length 4 because 3 is missing.

We need to write a method isAllPossibilities(int[] arr) that:

- Returns 1 if the array is valid.
- Returns 0 otherwise.

Code:

```
package com.mycompany.kopilaisallpossibilities;
public class Kopilaisallpossibilities {
    public static int isAllPossibilities(int[] arr) {
    int n = arr.length;
    if (n == 0) {
      return 0;   }
    for (int i = 0; i < n; i++) {
      boolean found = false;
      for (int j = 0; j < n; j++) {
        if (arr[j] == i) {
          found = true;
          break;     }}
      if (!found) {
        return 0;  } }
    return 1;    }
  public static void main(String[] args) {
    int[] arr1 = {1, 2, 0, 3};
    int[] arr2 = {0, 1, 2, 4};
    System.out.println("possibility of array 1 is: " +isAllPossibilities(arr1));
    System.out.println("possibility of array 2 is: " +isAllPossibilities(arr2));   }}
```

Output:

```
ut - Run (kopilaisallpossibilities)
--- exec:3.1.0:exec (default-cli) @ kopilaisallpossibiliti
possibility of array 1 is: 1
possibility of array 2 is: 0
------------------------------------------------------------
BUILD SUCCESS
```

23.Write a Java program that finds sum of two and three numbers using concept of method overloading.

Introduction: Method Overloading in Java means defining multiple methods with the same name but with different parameter lists. The compiler decides which method to call based on the number or type of arguments.

Syntax:

```
class ClassName {
   returnType methodName(parameter1, parameter2) { ... }
   returnType methodName(parameter1, parameter2, parameter3) { ... }
}
```

Code:

```
package com.mycompany.kopilamethodoverloading;
public class Kopilamethodoverloading {
   public int sum(int a,int b){
      return a+b;    }
   public int sum(int a,int b,int c){
      return a+b+c;    }
   public static void main(String[] args){
      Kopilamethodoverloading obj=new Kopilamethodoverloading();
      System.out.println("Sum of two numbers: "+obj.sum(10,20));
      System.out.println("Sum of three numbers: "+obj.sum(5,15,25));   }}
```

Output:

```
com.mycompany.kopilamethodoverloading.Kopilamethodoverloading
put - Run (kopilamethodoverloading)
--- exec:3.1.0:exec (default-cli) @ kopilamethodoverloa
Sum of two numbers: 30
Sum of three numbers: 45
------------------------------------------------------------
BUILD SUCCESS
```

24.Write a Java program that finds areas of different geometric shapes using concept of method overloading.

Method Overloading in Java allows us to create multiple methods with the same name but with different parameter lists. Here, we will use it to calculate areas of different geometric shapes (circle, rectangle, triangle, square, etc.).

Code:

```
package com.mycompany.kopilaareageometric;

public class Kopilaareageometric {

   public double area(double radius){

      return Math.PI*radius*radius;  }

   public double area(double length,double breadth){

      return length*breadth;    }

   public double area(int side){

      return side*side;    }

   public double area(double base,double height,boolean isTriangle){

      return 0.5*base*height;    }

   public static void main(String[] args){

      Kopilaareageometric obj=new Kopilaareageometric();

      System.out.println("Area of Circle: "+obj.area(7.0));

      System.out.println("Area of Rectangle: "+obj.area(10.0,5.0));

      System.out.println("Area of Square: "+obj.area(6));

      System.out.println("Area of Triangle: "+obj.area(8.0,4.0,true));   }}
```

```
com.mycompany.kopilaareageometric.Kopilaareageom
out - Run (kopilaareageometric)
--- exec:3.1.0:exec (default-cli) @ kopilaa
Area of Circle: 153.9380400258985
Area of Rectangle: 50.0
Area of Square: 36.0
Area of Triangle: 16.0
------------------------------------------
BUILD SUCCESS
```

25.Create a class Number with three int instance variable x , y and z. The class will have one constructor. The class also will contain member function getMax () that will return the largest number. Create a main method that will create an object of Number and will print the largest number.

Code:

```
package com.mycompany.kopilagetmax;
```

```
public class Kopilagetmax {
    int x, y, z;
    public Kopilagetmax(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;    }
    public int getMax() {
        int max = x;
        if (y > max) {
            max = y;     }
        if (z > max) {
            max = z;        }
        return max;    }
       public static void main(String[] args) {
        Kopilagetmax num = new Kopilagetmax(12, 45, 30);
        System.out.println("Largest number is: " + num.getMax());    }}
```

Output:

```
: - Run (kopilagetmax)  ×

--- exec:3.1.0:exec (default-cli) @ kopilaget:
Largest number is: 45
------------------------------------------
BUILD SUCCESS
```

26.Write a Java program to add two complex numbers.

Theory :

A **complex number** is a number in the form a + bi, where a is the real part and b is imaginary.
For addition : (a+bi)+(c+di)=(a+c)+(b+d)i
So, we simply add the real parts and the imaginary parts separately.

Code:

```
package com.mycompany.kopilacomplexnoadd;
class Complex {
   int real, imag;
   Complex(int r, int i) {
       real = r;
       imag = i;    }
   Complex add(Complex c) {
       return new Complex(real + c.real, imag + c.imag);    }
   String display() {
```

```
    return real + " + " + imag + "i";    }}
public class Kopilacomplexnoadd {
    public static void main(String[] args) {
     Complex c1 = new Complex(3, 2);
     Complex c2 = new Complex(4, 5);
     Complex sum = c1.add(c2);
     System.out.println("Sum = " + sum.display());    }}
```

Output:

```
- Run (kopilacomplexnoadd)  ×

--- exec:3.1.0:exec (default-cli) @ kopilacomplexno
Sum = 7 + 7i
------------------------------------------------------
BUILD SUCCESS
```

27.Write a Java program to add two Time(hr,min,sec) objects.

Theory: A Time object can have hours, minutes, and seconds. To add two times:

- Sum the seconds, minutes, and hours separately.
- If seconds ≥ 60, convert extra seconds to minutes.
- If minutes ≥ 60, convert extra minutes to hours.

Syntax: 1.Create a Time class with hr, min, sec.

2.Create a method addTime(Time t) to return the sum.

3.Display in main().

Code:

```
package com.mycompany.kopilaaddtime;
class Time {
   int hr, min, sec;
   Time(int h, int m, int s) {
      hr = h;
      min = m;
      sec = s;   }
   Time addTime(Time t) {
      int s = sec + t.sec;
      int m = min + t.min + s / 60;
      int h = hr + t.hr + m / 60;
      s %= 60;
      m %= 60;
      return new Time(h, m, s);    }
```

```
    String display() {
        return hr + "hr " + min + "min " + sec + "sec";   }}
public class Kopilaaddtime {
    public static void main(String[] args) {
        Time t1 = new Time(2, 45, 50);
        Time t2 = new Time(1, 20, 15);
        Time sum = t1.addTime(t2);
        System.out.println("Sum of time: " + sum.display());   }}
```

Output:

```
: - Run (kopilaaddtime)  ×

--- exec:3.1.0:exec (default-cli) @ kopilaaddtime
Sum of time: 4hr 6min 5sec
------------------------------------------------
BUILD SUCCESS
```

28. Create a class Swapper class with two integer instance variable x and y and constructor with two parameters that initializes the two variables. Also include three member functions: A getX () that returns x, a getY () function that returns y, a void swap () method that swaps the values of x and y. Then define a main() method to create an object of Swapper class and swap the value of instance variables.

Introduction: A Swapper class demonstrates object-oriented programming concepts like encapsulation and methods. It contains instance variables and member functions to manipulate data, specifically to swap two numbers.

- x and y are integer instance variables.Constructor initializes x and y.
- getX() and getY() return the values.
- swap() exchanges the values of x and y.

Code:

```
package com.mycompany.kopilagetswap;
class Swapper {
  int x, y;
  Swapper(int a, int b) {
    x = a;
    y = b;   }
  int getX() {
    return x;   }
  int getY() {
```

```
        return y;    }
    void swap() {
        int temp = x;
        x = y;
        y = temp;   }}
public class Kopilagetswap {
    public static void main(String[] args) {
        Swapper obj = new Swapper(10, 20);
        System.out.println("Before swap: x=" + obj.getX() + " y=" + obj.getY());
        obj.swap();
        System.out.println("After swap: x=" + obj.getX() + " y=" + obj.getY());    }}
```

Output:

```
ut - Run (kopilagetswap)
--- exec:3.1.0:exec (default-cli) @ kopilagetswap
Before swap: x=10 y=20
After swap: x=20 y=10
------------------------------------------------
BUILD SUCCESS
```

29. Create a class Date with three integer instance variables named day, month, year. It has a constructor with three parameters for initializing the instance variables, and it has one-member function named daySinceJan1 (). It computes and returns the number of days since January 1 of the same year, including January 1 and the day in the Date object. For example, if day is a Date object with day = 1, month = 3 and year = 2000, then the call date.daySinceJan1() should return 61 since there are 61 days between the dates of January 1, 2000, and March 1, 2000, including January 1 and March 1. Then define main () method to handle Date class. Don't forget leap years.

Introduction: The Date class demonstrates how to work with instance variables, methods, and date calculations. It calculates the number of days since January 1 of the same year, taking leap years into account.

- day, month, year are integer instance variables.
- Constructor initializes the date.
- daySinceJan1() calculates the total days from January 1 to the date in the object.
- Leap years are considered:
    - A year is a leap year if divisible by 4 and (not divisible by 100 or divisible by 400).

Code:

```
package com.mycompany.kopiladatetime;
```

```
class Date {
    int day, month, year;
    Date(int d, int m, int y) {
        day = d;
        month = m;
        year = y;    }
    boolean isLeapYear() {
        return (year % 4 == 0 && (year % 100 != 0 || year % 400 == 0));    }
    int daySinceJan1() {
        int[] daysInMonth = {31,28,31,30,31,30,31,31,30,31,30,31};
        if(isLeapYear()) daysInMonth[1] = 29;
        int totalDays = 0;
        for(int i = 0; i < month - 1; i++) totalDays += daysInMonth[i];
        totalDays += day;
        return totalDays;    }}
public class Kopiladatetime {
    public static void main(String[] args) {
        Date date = new Date(1,3,2000);
        System.out.println("Days since Jan 1: " + date.daySinceJan1());   }}
```

Output:

```
it - Run (kopiladatetime)  ×

] --- exec:3.1.0:exec (default-cli) @ kopiladatetime
- Days since Jan 1: 61
  ------------------------------------------------------------
  BUILD SUCCESS
```

30.Create a USMoney class with two integer instance variables dollars and cents. Add a constructor with two parameters for initializing a USMoney object. The constructor should check that the cent value is between 0 and 99 and, if not, transfer some cents to the dollars variables to make it between 0 and 99. For example, if x is a USMoney object with 5 dollars and 80 cents, and if y is a USMoney object with 1 dollar and 90 cents, then x.plus (y) will return a new USMoney object with 7 dollars and 70 cents. Also, create a main () method that creates to objects of USMoney class and add them.

Introduction: The USMoney class represents money in dollars and cents. It demonstrates constructors, instance variables, and methods to perform addition of money.

- dollars and cents are integer instance variables.
- Constructor ensures cents is between 0–99 by converting extra cents into dollars.
- Method plus(USMoney m) adds two USMoney objects and returns a new USMoney object with normalized cents.

Code:

```
package com.mycompany.kopilausmoney;
class USMoney {
   int dollars, cents;
   USMoney(int d, int c) {
      dollars = d + c / 100;
      cents = c % 100;    }
   USMoney plus(USMoney m) {
      int totalCents = cents + m.cents;
      int totalDollars = dollars + m.dollars + totalCents / 100;
      totalCents %= 100;
      return new USMoney(totalDollars, totalCents);    }
   String display() {
      return dollars + " dollars and " + cents + " cents";   }}
public class Kopilausmoney {
   public static void main(String[] args) {
      USMoney x = new USMoney(5, 80);
      USMoney y = new USMoney(1, 90);
      USMoney sum = x.plus(y);
      System.out.println("Sum: " + sum.display());    }}
```

Output:

```
t - Run (kopilausmoney)  ×

] --- exec:3.1.0:exec (default-cli) @ kopilausmoney
- Sum: 7 dollars and 70 cents
  ------------------------------------------------
  BUILD SUCCESS
```

31.Create a Person class with private instance variables for person's name and birth date. Add appropriate functions for these variables. Then create a subclass CollegeGraduate with private instance variables for the student's GPA and year of graduation and appropriate functions for these variables. Don't forget to include

appropriate constructor constructors for your classes. Then define main () method that demonstrates your classes.

Introduction: The Person class demonstrates encapsulation using private instance variables and getter/setter methods. The CollegeGraduate subclass extends Person to include additional attributes like GPA and year of graduation.

- Person has name and birthDate (private).
- CollegeGraduate extends Person and adds GPA and yearOfGraduation.
- Constructors initialize all fields.
- Getter and setter methods provide controlled access to private variables.

Code:

```
package com.mycompany.kopilapersoninfo;
    class Person {
        private String name, birthDate;
        Person(String n, String b) {
            name = n;
            birthDate = b;    }
        String getName() {
            return name;    }
        void setName(String n) {
            name = n;    }
        String getBirthDate() {
            return birthDate;    }
        void setBirthDate(String b) {
            birthDate = b;    }}
    class CollegeGraduate extends Person {
        private double gpa;
        private int yearOfGraduation;
        CollegeGraduate(String n, String b, double g, int y) {
            super(n, b);
            gpa = g;
            yearOfGraduation = y;    }
        double getGPA() {
            return gpa;    }
        void setGPA(double g) {
```

```
        gpa = g;    }
    int getYearOfGraduation() {
        return yearOfGraduation;    }
    void setYearOfGraduation(int y) {
        yearOfGraduation = y;    }}
public class Kopilapersoninfo {
    public static void main(String[] args) {
        CollegeGraduate student = new CollegeGraduate("Alice", "2000-05-15", 3.8, 2024);
        System.out.println("Name: " + student.getName());
        System.out.println("Birth Date: " + student.getBirthDate());
        System.out.println("GPA: " + student.getGPA());
        System.out.println("Year of Graduation: " + student.getYearOfGraduation());    }}
```

Output:

```
tput - Run (kopilapersoninfo)
--- exec:3.1.0:exec (default-cli) @ kopilapersoninfo ---
Name: Alice
Birth Date: 2000-05-15
GPA: 3.8
Year of Graduation: 2024
------------------------------------------------------------
BUILD SUCCESS
```

32.Create a class Box with fields width, height and depth. Add methods getArea () and getVolume (). Use suitable constructors. From main () method create an object of Box class and find its area as volume.

Introduction

The **Box class** represents a three-dimensional box and demonstrates **constructors, methods, and object-oriented concepts**. It calculates **surface area** and **volume** of the box.

- getArea() calculates the **surface area**: 2*(wh + hd + wd).
- getVolume() calculates the **volume**: width * height * depth.

Code:

```
package com.mycompany.kopilaboxclass;
    class Box {
        int width, height, depth;
        Box(int w, int h, int d) {
            width = w;
            height = h;
            depth = d;    }
```

```
    int getArea() {
        return 2 * (width * height + height * depth + width * depth);    }
    int getVolume() {
        return width * height * depth;    }}
  public class Kopilaboxclass {
    public static void main(String[] args) {
        Box box = new Box(3, 4, 5);
        System.out.println("Surface Area: " + box.getArea());
        System.out.println("Volume: " + box.getVolume());    }}
```

Output:

```
ut - Run (kopilaboxclass)
--- exec:3.1.0:exec (default-cli) @ kopilaboxclass
Surface Area: 94
Volume: 60
------------------------------------------------------------
BUILD SUCCESS
```

33. Create a class Room with instance variables length and breadth. Add one function getArea () that returns the area of the room. Create a subclass MyRoom and add one instance variable height. Add one function getVolume () that returns the volume. Then define main () method that creates two MyRoom objects and find area and volumes of both rooms.

Code:

```
package com.mycompany.kopilaroomarea;
class Room {
  int length, breadth;
  Room(int l, int b) {
    length = l;
    breadth = b;    }
  int getArea() {
    return length * breadth;    }}
class MyRoom extends Room {
  int height;
  MyRoom(int l, int b, int h) {
    super(l, b);
    height = h;    }
  int getVolume() {
```

```
        return length * breadth * height;    }}
public class Kopilaroomarea {
    public static void main(String[] args) {
        MyRoom room1 = new MyRoom(5, 4, 3);
        MyRoom room2 = new MyRoom(6, 5, 4);
        System.out.println("Room1 Area: " + room1.getArea());
        System.out.println("Room1 Volume: " + room1.getVolume());
        System.out.println("Room2 Area: " + room2.getArea());
        System.out.println("Room2 Volume: " + room2.getVolume());    }}
```

Output:

```
- Run (kopilaroomarea)
--- exec:3.1.0:exec (default-cli) @ kopilaroomarea
Room1 Area: 20
Room1 Volume: 60
Room2 Area: 30
Room2 Volume: 120
------------------------------------------------------------
BUILD SUCCESS
```

34.Create a class Box with instance variables length, breadth and height. Add one method getVolume () to compute the volume of box. Use suitable constructors. Create a subclass BoxWeight that extends Box that add one variable weight. Add one function getWeight () that displays the weight of box to this class. Add suitable constructors. Create one more subclass class Shipment that extends BoxWeight. Add one function getCost () that displays the cost of the box. Add suitable constructors. Then define main () method that creates an object of Shipment that initializes the instance variables through constructor.

Introduction

This program demonstrates **multilevel inheritance** and the use of **member functions** to access object data:

- getVolume() in Box calculates the volume of the box.
- getWeight() in BoxWeight displays the weight of the box.
- getCost() in Shipment displays the cost associated with shipping the box.

Each function is specific to the class it belongs to, showing how methods in subclasses can access both inherited and new data.

Code:

```
package com.mycompany.kopilaboxwtshipment;

class Box {
    int length, breadth, height;
    Box(int l, int b, int h) {
        length = l;
        breadth = b;
        height = h;    }
```

```
    int getVolume() {
        return length * breadth * height;    }}
class BoxWeight extends Box {
    int weight;
    BoxWeight(int l, int b, int h, int w) {
        super(l, b, h);
        weight = w;    }
    void getWeight() {
        System.out.println("Weight: " + weight + " kg");    }}
class Shipment extends BoxWeight {
    int cost;
    Shipment(int l, int b, int h, int w, int c) {
        super(l, b, h, w);
        cost = c;    }
    void getCost() {
        System.out.println("Cost: $" + cost);    }}
public class Kopilaboxwtshipment {
    public static void main(String[] args) {
        Shipment s = new Shipment(5, 4, 3, 10, 150);
        System.out.println("Volume: " + s.getVolume());
        s.getWeight();
        s.getCost();    }}
```

Output:

```
ut - Run (kopilaboxwtshipment)  ×

--- exec:3.1.0:exec (default-cli) @ kopilaboxwtshipment
Volume: 60
Weight: 10 kg
Cost: $150
```

35.Create an abstract class Figure with two instance variables dim1 and dim2. Add suitable constructors. Add one abstract function called getArea (). Create two subclass called Rectangle and Triangle. Add function getArea () to both of the classes that will find the area of respective figures. Then define main () method that creates an object of each classes and find the area of triangle and rectangle.

Theory: This program demonstrates **abstract classes** and **method overriding** in Java:

- An **abstract class** is a class that cannot be instantiated and can contain abstract methods (methods without a body). Subclasses must provide implementation for abstract methods.
- **Method overriding** occurs when a subclass provides its own implementation of a method declared in a superclass.
- Figure is an abstract class with dim1 and dim2, and an abstract method getArea().
- Rectangle and Triangle are subclasses that **override getArea()** to calculate the area of their respective shapes.

Code:

```
package com.mycompany.kopilafigureclass;
```

```
abstract class Figure {
    double dim1, dim2;
    Figure(double d1, double d2) {
        dim1 = d1;
        dim2 = d2;    }
    abstract double getArea();}
class Rectangle extends Figure {
    Rectangle(double l, double b) {
        super(l, b);    }
    double getArea() {
        return dim1 * dim2;    }}
class Triangle extends Figure {
    Triangle(double b, double h) {
        super(b, h);    }
    double getArea() {
        return 0.5 * dim1 * dim2;    }}
public class Kopilafigureclass {
    public static void main(String[] args) {
        Rectangle rect = new Rectangle(5, 4);
        Triangle tri = new Triangle(6, 3);
        System.out.println("Rectangle Area: " + rect.getArea());
        System.out.println("Triangle Area: " + tri.getArea());    }}
```

Output:

```
: - Run (kopilafigureclass)
--- exec:3.1.0:exec (default-cli) @ kopilafigureclass
Rectangle Area: 20.0
Triangle Area: 9.0
------------------------------------------------------
BUILD SUCCESS
```

36. Write a Java program to demonstrate divide by zero exception.

Introduction: A **divide by zero exception** occurs when a program attempts to divide a number by zero. In Java, this triggers an **ArithmeticException**. Handling it with a **try-catch block** prevents the program from crashing.

Syntax:

```
try {
    // code that may cause exception
} catch (ArithmeticException e) {
    // handle exception
}
```

Code:

```
package com.mycompany.kopiladividebyzeo;
public class Kopiladividebyzeo {
    public static void main(String[] args) {
        int a = 10, b = 0;
        try {
            int result = a / b;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");     }}}
```

Output:

```
put - Run (kopiladividebyzeo)
--- exec:3.1.0:exec (default-cli) @ kopiladividebyzeo
Error: Division by zero is not allowed.
------------------------------------------------------------
BUILD SUCCESS
```

37.Write a Java program to demonstrate array index bounds exception.

Introduction: An **Array Index Out of Bounds Exception** occurs when a program tries to access an array element with an index that is either negative or greater than or equal to the array size. Handling it with a **try-catch block** prevents the program from crashing.

Syntax:

```
try {
    // access array element
} catch (ArrayIndexOutOfBoundsException e) {
    // handle exception}
```

Code:

```
package com.mycompany.kopilaarrayoutofbound;
```

```
public class Kopilaarrayoutofbound {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        try {
            System.out.println(arr[5]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Array index out of bounds.");   }}}
```

Output:

```
com.mycompany.kopilaarrayoutofbound.Kopilaarrayoutofbound
Output - Run (kopilaarrayoutofbound)
--- exec:3.1.0:exec (default-cli) @ kopilaarrayoutofbound
Error: Array index out of bounds.
------------------------------------------------------------
BUILD SUCCESS
```

38.Write a Java Program to demonstrate null exception.

Introduction: A **Null Pointer Exception** occurs when a program attempts to use an object reference that has not been initialized (i.e., points to null). Handling it with a **try-catch block** prevents the program from crashing.

Code:

```
package com.mycompany.kopilanullpointer;
public class Kopilanullpointer {
    public static void main(String[] args) {
        String str = null;
        try {
            System.out.println(str.length());
        } catch (NullPointerException e) {
            System.out.println("Error: Null reference encountered.");    }}}
```

Output:

```
Output - Run (kopilanullpointer)
--- exec:3.1.0:exec (default-cli) @ kopilanullpointer
Error: Null reference encountered.
------------------------------------------------------------
BUILD SUCCESS
```

39. Write a Java program to demonstrate multiple catch statements.

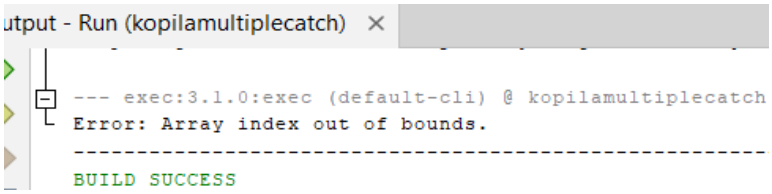
Introduction: In Java, a **try block** can be followed by **multiple catch blocks** to handle different types of exceptions separately. This allows a program to handle **different errors** gracefully.

- Each **catch block** specifies a different exception type.
- Java executes the **first matching catch block** for the thrown exception.

Code:

```
package com.mycompany.kopilamultiplecatch;
    public class Kopilamultiplecatch {
        public static void main(String[] args) {
            int[] arr = {1, 2, 3};
            int a = 10, b = 0;
            try {
                System.out.println(arr[5]);
                int result = a / b;
                System.out.println("Result: " + result);
            } catch (ArithmeticException e) {
                System.out.println("Error: Division by zero.");
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Error: Array index out of bounds.");    }}}
```

Output:



40. Write a Java program to demonstrate custom exception

Introduction: A **custom exception** is a user-defined exception that extends the Exception class. It allows you to create meaningful exception types for specific error conditions in your program.

Syntax:

```
class MyException extends Exception {
   MyException(String message) {
      super(message);   }}
```

```
try {
    // code that may throw MyException
} catch (MyException e) {
    // handle custom exception
}
```

Code:

```
package com.mycompany.kopilacustomexception;
class NegativeNumberException extends Exception {
    NegativeNumberException(String message) {
        super(message);    }}
public class Kopilacustomexception{
    static void checkNumber(int n) throws NegativeNumberException {
        if (n < 0) {
            throw new NegativeNumberException("Negative numbers are not allowed: " + n);
        } else {
            System.out.println("Number is valid: " + n);        }}
    public static void main(String[] args) {
        try {
            checkNumber(5);
            checkNumber(-10);
        } catch (NegativeNumberException e) {
            System.out.println("Custom Exception: " + e.getMessage());  }}}
```

Output:

```
:put - Run (kopilacustomexception)
--- exec:3.1.0:exec (default-cli) @ kopilacustomexception
Number is valid: 5
Custom Exception: Negative numbers are not allowed: -10
------------------------------------------------------------
BUILD SUCCESS
```

41.Write a Java program to concatenate two strings.

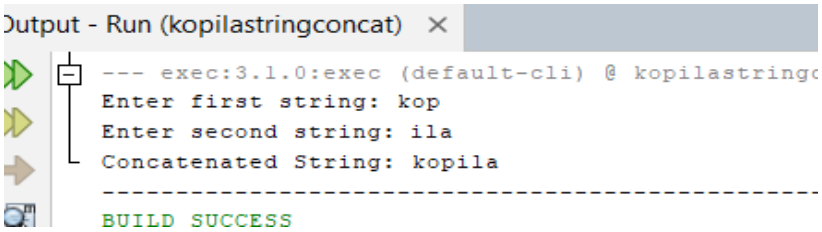
Introduction: String concatenation in Java means combining two or more strings into a single string. It can be done using the **+ operator** or the **concat() method**.

Code:

```
package com.mycompany.kopilastringconcat;
    import java.util.Scanner;
```

```
public class Kopilastringconcat {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first string: ");
        String str1 = sc.nextLine();
        System.out.print("Enter second string: ");
        String str2 = sc.nextLine();
        String result = str1 + str2;
        System.out.println("Concatenated String: " + result);  }}
```

Output:



42.Write a Java program to compare two strings.

In Java, strings can be compared in multiple ways:

- **equals() method**: Checks if two strings are exactly equal (case-sensitive).
- **equalsIgnoreCase() method**: Checks equality ignoring case.
- **compareTo() method**: Compares lexicographically, returns 0 if equal, negative if first is smaller, positive if first is greater.

Code:

```
package com.mycompany.kopilastringcompare;
import java.util.Scanner;
public class Kopilastringcompare{
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first string: ");
      String str1 = sc.nextLine();
      System.out.print("Enter second string: ");
      String str2 = sc.nextLine();
      if (str1.equals(str2)) {
```

```
        System.out.println("Strings are equal (case-sensitive).");
    } else {
        System.out.println("Strings are not equal (case-sensitive).");   }
    if (str1.equalsIgnoreCase(str2)) {
        System.out.println("Strings are equal (ignoring case).");
    } else {
        System.out.println("Strings are not equal (ignoring case)."); }
    int cmp = str1.compareTo(str2);
    if (cmp == 0) {
        System.out.println("Strings are equal lexicographically.");
    } else if (cmp < 0) {
        System.out.println("First string is smaller lexicographically.");
    } else {
        System.out.println("First string is greater lexicographically."); }}}
```

Output:

```
tput - Run (kopilastringcompare)
--- exec:3.1.0:exec (default-cli) @ kopilastri
Enter first string: kopila
Enter second string: kamal
Strings are not equal (case-sensitive).
Strings are not equal (ignoring case).
First string is greater lexicographically.
------------------------------------------------
BUILD SUCCESS
```

43.Write a Java program to compare some portion of two Strings.

In Java, you can compare parts of strings using the regionMatches() method:

- Compares a specified portion of one string with a portion of another string.
- Can be case-sensitive or case-insensitive.

Syntax:

str1.regionMatches(startIndex1, str2, startIndex2, length)

str1.regionMatches(ignoreCase, startIndex1, str2, startIndex2, length)

Code:

```
package com.mycompany.kopilastringsomeportion;
import java.util.Scanner;
```

```
public class  Kopilastringsomeportion {
   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      System.out.print("Enter first string: ");
      String str1 = sc.nextLine();
      System.out.print("Enter second string: ");
      String str2 = sc.nextLine();
      boolean match = str1.regionMatches(0, str2, 0, Math.min(str1.length(), str2.length()));
      if (match) {
         System.out.println("The beginning portions of both strings match.");
      } else {
         System.out.println("The beginning portions of both strings do not match.");   }}}
```

Output:

```
ıt - Run (kopilastringsomeportion)
Enter first string: abc
Enter second string: abcd
The beginning portions of both strings match.
------------------------------------------------
BUILD SUCCESS
```

44. Write a Java program that replaces some characters in Strings.

Introduction: In Java, strings are immutable objects, meaning their content cannot be changed once created. However, Java provides built-in methods to create a new modified string.

- To **replace characters**, we use the **replace()** method of the String class.
- It returns a new string with all occurrences of the old character (or substring) replaced with the new one.

Syntax:

String newString = oldString.replace(oldChar, newChar);

OR, String newString = oldString.replace(oldSubstring, newSubstring);

Code:

```
package com.mycompany.kopilastringreplace;
import java.util.Scanner;
public class Kopilastringreplace {
   public static void main(String[] args) {
```

```
Scanner sc = new Scanner(System.in);
System.out.print("Enter a string: ");
String str = sc.nextLine();
System.out.print("Enter the character to replace: ");
char oldChar = sc.next().charAt(0);
System.out.print("Enter the new character: ");
char newChar = sc.next().charAt(0);
String newStr = str.replace(oldChar, newChar);
System.out.println("Original String: " + str);
System.out.println("Modified String: " + newStr);   }}
```

Output:

```
utput - Run (kopilastringreplace)
--- exec:3.1.0:exec (default-cli) @ kopilastringreplace ---
Enter a string: abc
Enter the character to replace: b
Enter the new character: p
Original String: abc
Modified String: apc
------------------------------------------------------------
BUILD SUCCESS
```

45. Write a Java program to reads contents of a file using character stream.

Introduction: In Java, **character streams** are used to read and write text files (characters). Unlike byte streams, character streams are designed for handling characters and automatically handle character encoding.

- To **read characters from a file**, we commonly use the **FileReader** class.
- It reads one character at a time and returns it as an integer.
- If the end of the file is reached, it returns -1.

Syntax:

```
FileReader fr = new FileReader("filename.txt");
int i;
while((i = fr.read()) != -1) {
   System.out.print((char)i);}
fr.close();
```

Code:

```
package com.mycompany.kopilafilereader;
import java.io.FileReader;
```

```
import java.io.IOException;
public class Kopilafilereader {
   public static void main(String[] args) {
      try {
          FileReader fr = new FileReader("textfile.txt");
         int i;
         System.out.println("File contents:");
         while ((i = fr.read()) != -1) {
            System.out.print((char) i);    }
         fr.close();
      } catch (IOException e) {
         System.out.println("Error reading file: " + e.getMessage());  }}}
```

Output:

```
com.mycompany.kopilafilereader.Kopilafilereader >  main >  try
put - Run (kopilafilereader)
--- exec:3.1.0:exec (default-cli) @ kopilafilereader ---
File contents:
Hi ! I'm Kopila. This is file to read character steam.
------------------------------------------------------------
BUILD SUCCESS
```

46. Write a Java program to write some lines of text to file using character stream.

Introduction: In Java, we can write characters to a file using the **FileWriter** class (a character stream). **Syntax:** FileWriter writer = new FileWriter("filename.txt");

writer.write("Some text");

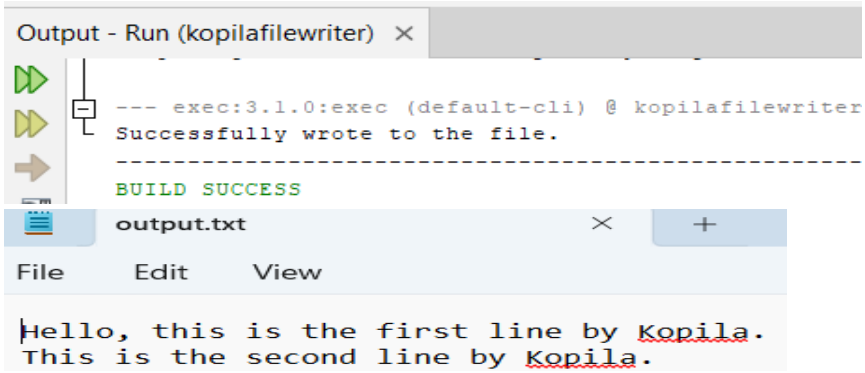
writer.close();

Code:

```
package com.mycompany.kopilafilewriter;
import java.io.FileWriter;
import java.io.IOException;
public class Kopilafilewriter {
   public static void main(String[] args) {
      try {
         FileWriter writer = new FileWriter("output.txt");
         writer.write("Hello, this is the first line by Kopila.\n");
         writer.write("This is the second line by Kopila.\n");
```

```
        writer.close();
        System.out.println("Successfully wrote to the file.");
    } catch (IOException e) {
        System.out.println("An error occurred while writing the file.");
        e.printStackTrace();    }}}
```

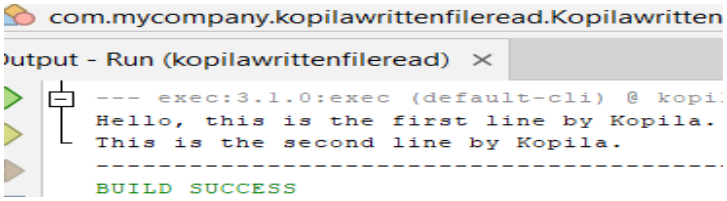
Output:



47.Write a Java program that reads contents of same file using character stream.

Code:

```
package com.mycompany.kopilawrittenfileread;

import java.io.FileReader;

import java.io.IOException;

public class Kopilawrittenfileread {

public static void main(String[] args) {

try { FileReader reader = new
FileReader("C:/Users/devko/OneDrive/Documents/NetBeansProjects/kopilafilewriter/output.txt");
        int ch;

        while ((ch = reader.read()) != -1) {

            System.out.print((char) ch);   }

        reader.close();

    } catch (IOException e) {

        System.out.println("An error occurred while reading the file.");

        e.printStackTrace();      }}}
```



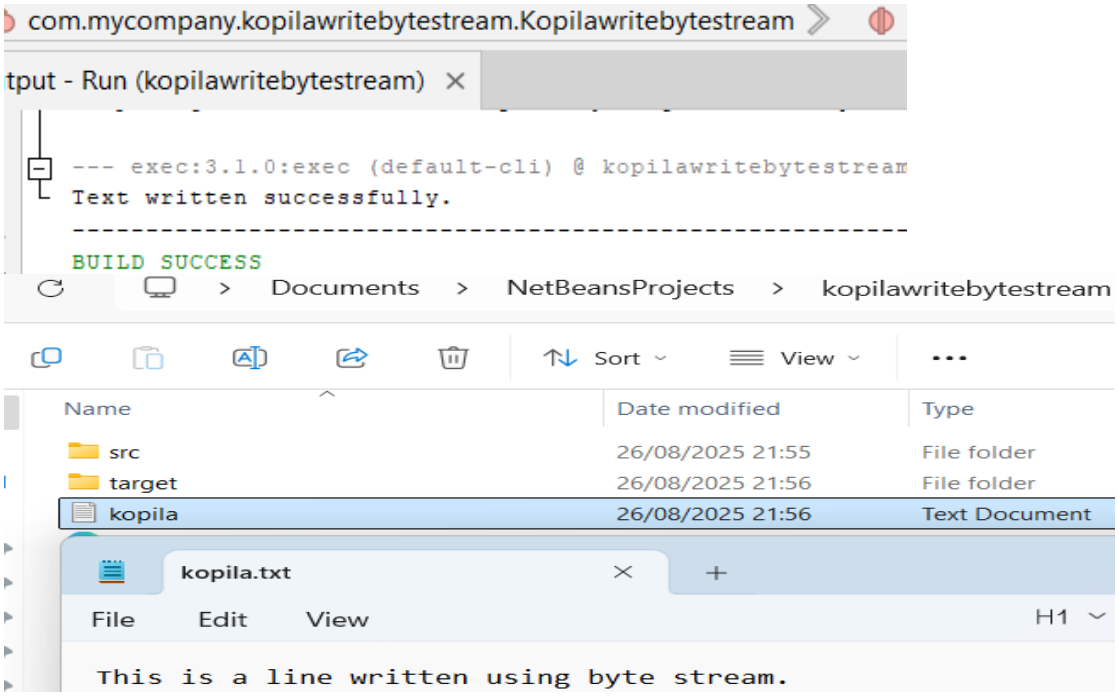
48.Write a Java program that writes line of text to file using byte stream.

Theory: In Java, byte streams are used to handle raw binary data. The FileOutputStream class is commonly used to write bytes to a file. Each character/string is converted to bytes before writing.

Code:

```
package com.mycompany.kopilawritebytestream;

import java.io.FileOutputStream;

import java.io.IOException;

public class Kopilawritebytestream {

   public static void main(String[] args) {

      String text = "This is a line written using byte stream.";

      try {

         FileOutputStream out = new FileOutputStream("output.txt");

         out.write(text.getBytes());

         out.close();

         System.out.println("Text written successfully.");

      } catch (IOException e) {

         System.out.println("An error occurred while writing the file.");

         e.printStackTrace();     }}}
```

Output:



49. Write a Java program that reads contents of file using byte stream.

Introduction: In Java, byte streams are used to handle raw binary data.
The FileInputStream class reads bytes from a file. Each byte can be converted to a character for text files.

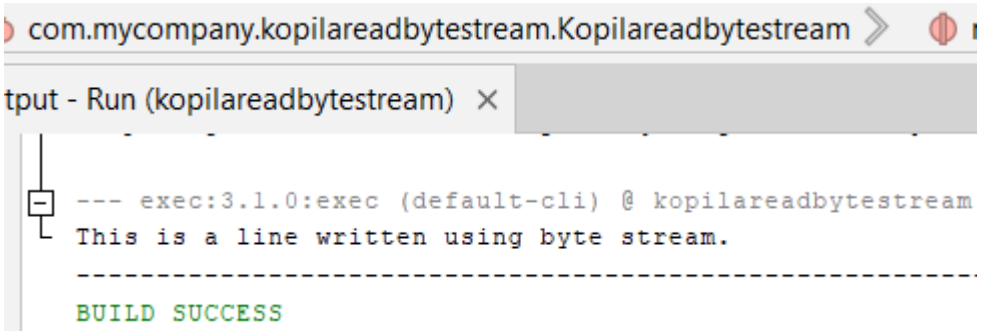
Syntax:

```
FileInputStream in = new FileInputStream("filename.txt");
int b;
while ((b = in.read()) != -1) {
   // process byte}
in.close();
```

Code:

```
package com.mycompany.kopilareadbytestream;
import java.io.FileInputStream;
import java.io.IOException;
public class Kopilareadbytestream{
   public static void main(String[] args) {
      try {
      FileInputStream in = new
FileInputStream("C:\\Users\\devko\\OneDrive\\Documents\\NetBeansProjects\\kopilawritebytestream
\\kopila.txt");
         int b;
         while ((b = in.read()) != -1) {
            System.out.print((char) b); }
         in.close();
      } catch (IOException e) {
         System.out.println("An error occurred while reading the file.");
         e.printStackTrace();   }}}
```

Output:



50.Write a Java program to write objects to file.

Introduction: In Java, Object streams are used to read/write entire objects to/from files. The ObjectOutputStream class writes objects to a file, while ObjectInputStream reads them back. Objects must implement the Serializable interface.

Code:

```
package com.mycompany.kopilawriteobject;

import java.io.FileWriter;

import java.io.IOException;

import java.io.PrintWriter;

class Student {

   String name;

   int rollNo;

   Student(String name, int rollNo) {

      this.name = name;

      this.rollNo = rollNo;   }}

 public class Kopilawriteobject {

    public static void main(String[] args) {

      Student s1 = new Student("Sujan", 101);

      Student s2 = new Student("Kopila", 102);

      try {

         PrintWriter pw = new PrintWriter(new FileWriter("students.txt"));

         pw.println(s1.name + "," + s1.rollNo);

         pw.println(s2.name + "," + s2.rollNo);

         pw.close();

         System.out.println("Student data written to students.txt successfully.");

      } catch (IOException e) {

         System.out.println("Error writing to file.");

         e.printStackTrace();   }}}
```

```
com.mycompany.kopilawriteobject.Kopilawriteobject
tput - Run (kopilawriteobject)

--- exec:3.1.0:exec (default-cli) @ kopilawriteobj
Student data written to students.txt successfully.
------------------------------------------------------------
BUILD SUCCESS
```

51. Write a Java program to read objects from file.

Code:

```
package com.mycompany.kopilareadobject;
import java.io.FileReader;
import java.io.BufferedReader;
import java.io.IOException;
class Student {
   String name;
   int rollNo;
   Student(String name, int rollNo) {
      this.name = name;
      this.rollNo = rollNo;  }}
public class Kopilareadobject {
   public static void main(String[] args) {
      try {
         BufferedReader br = new BufferedReader(new
FileReader("C:\\Users\\devko\\OneDrive\\Documents\\NetBeansProjects\\kopilawriteobject\\students.
txt"));
         String line;
         while ((line = br.readLine()) != null) {
            String[] parts = line.split(",");
            Student s = new Student(parts[0], Integer.parseInt(parts[1]));
            System.out.println("Student: " + s.name + ", " + s.rollNo); }
         br.close();
      } catch (IOException e) {
         e.printStackTrace();
     } }
}
```

Output:

```
com.mycompany.kopilareadobject.Kopilareadobject > main
Output - Run (kopilareadobject)
--- exec:3.1.0:exec (default-cli) @ kopilareadobject
Student: Sujan, 101
Student: Kopila, 102
------------------------------------------------------------
BUILD SUCCESS
```

52.Write a Java program to demonstrate random access process .

Theory: The RandomAccessFile class in Java allows you to **read from and write to a file at any position**. You can move the file pointer using seek() method.

Syntax:

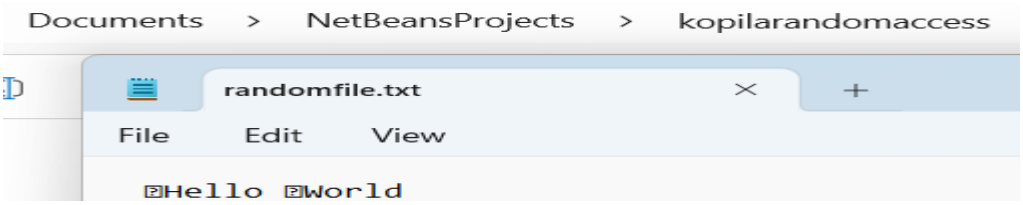
```
RandomAccessFile raf = new RandomAccessFile("filename", "mode");
raf.seek(position); // move pointer
raf.read();        // read data
raf.write();       // write data
raf.close();       // close file
```

Code:

```
package com.mycompany.kopilarandomaccess;
import java.io.RandomAccessFile;
import java.io.IOException;
public class Kopilarandomaccess {
    public static void main(String[] args) {
        try {
            RandomAccessFile raf = new RandomAccessFile("randomfile.txt", "rw");
            raf.writeUTF("Hello");
            long secondStringPos = raf.getFilePointer(); // store start position of second string
            raf.writeUTF("World");
            raf.seek(0);
            System.out.println("First String: " + raf.readUTF());
            raf.seek(secondStringPos);
            System.out.println("Second String: " + raf.readUTF());
            raf.close();
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());     }}}
```

Output:

```
utput - Run (kopilarandomaccess)
--- exec:3.1.0:exec (default-cli) @ kopilarandomaccess
First String: Hello
Second String: World
------------------------------------------------------------
BUILD SUCCESS
```



53.Write a Java program to print 10 random numbers between 1 to 100.

Theory: Random rand = new Random(); → creates a random number generator.

rand.nextInt(100) + 1 → generates numbers from 1 to 100.

for loop runs 10 times to print 10 numbers.

Code:

```
package com.mycompany.kopila1to100;

import java.util.Random;

public class Kopila1to100{

   public static void main(String[] args) {

      Random rand = new Random();

      for(int i = 0; i < 10; i++) {

         int num = rand.nextInt(100) + 1; // 1 to 100

         System.out.println(num);   }}}
```

Output:

54. Write a Java program that creates a class called Stack and then implements push() and pop() operations.

Description: A **stack** is a linear data structure that follows **LIFO (Last In First Out)** principle. The last element added is the first one to be removed.

- **push()**: Adds an element to the top of the stack.

- **pop()**: Removes and returns the element from the top of the stack.
- **isEmpty()**: Checks if the stack is empty.
- Stack has overflow (when adding to a full stack) and underflow (when removing from an empty stack) conditions.

Syntax (Class-based Implementation)

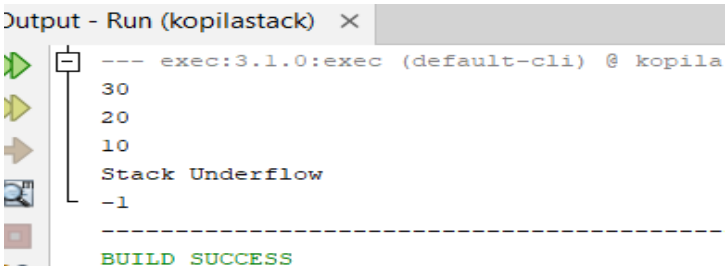
```
class Stack {
   private int maxSize; // maximum size of stack
   private int[] stackArray; // array to store elements
   private int top; // index of top element
   public Stack(int size) { } // constructor to initialize stack
   public void push(int value) { } // method to add element
   public int pop() { } // method to remove element
   public boolean isEmpty() { } // method to check if stack is empty
}
```

Code:

```
package com.mycompany.kopilastack;
class Stack {
   private int maxSize;
   private int[] stackArray;
   private int top;
   public Stack(int size) {
      maxSize = size;
      stackArray = new int[maxSize];
      top = -1;    }
   public void push(int value) {
      if (top == maxSize - 1) {
         System.out.println("Stack Overflow");
      } else {
         stackArray[++top] = value;      }}
   public int pop() {
      if (top == -1) {
         System.out.println("Stack Underflow");
         return -1;   } else {
```

```
        return stackArray[top--];    }}
    public boolean isEmpty() {
        return top == -1;   }}
public class Kopilastack {
    public static void main(String[] args) {
        Stack stack = new Stack(5);
        stack.push(10);
        stack.push(20);
        stack.push(30);
        System.out.println(stack.pop());
        System.out.println(stack.pop());
        System.out.println(stack.pop());
        System.out.println(stack.pop());    }}
```

Output:



55.Create an interface Exam with methods setExam(String division, int mark) and showExam(), create a class named test that implements the interface Exam and then display the records.

Description: An **interface** in Java defines a contract that a class can implement.

- **Exam** interface has methods setExam() to set exam details and showExam() to display them.
- **Test** class implements the interface and provides the behavior for these methods.

Code:

```
package com.mycompany.kopilaexaminterface;
        interface Exam {
            void setExam(String division, int mark);
            void showExam(); }
        class Test implements Exam {
            private String division;
```

```
        private int mark;
        public void setExam(String division, int mark) {
            this.division = division;
            this.mark = mark;    }
        public void showExam() {
            System.out.println("Division: " + division);
            System.out.println("Mark: " + mark);   }}
    public class Kopilaexaminterface {
        public static void main(String[] args) {
            Test t = new Test();
            t.setExam("A", 85);
            t.showExam();  }}
```

Output:

```
utput - Run (kopilaexaminterface)
--- exec:3.1.0:exec (default-cli
Division: A
Mark: 85
------------------------------
BUILD SUCCESS
```

56. Write a Java program to create a class Mobile (type, phone_no). Customize the exception such that if the user give phone_no having less than or greater than 10 digit, then the program has to throw an exception with message "Invalid Phone Number".

Code:

```
package com.mycompany.kopilamobileexception;
class InvalidPhoneNumberException extends Exception {
   public InvalidPhoneNumberException(String message) {
      super(message);    }}
class Mobile {
   private String type;
   private String phone_no;
   public Mobile(String type, String phone_no) throws InvalidPhoneNumberException {
      if (phone_no.length() != 10)
         throw new InvalidPhoneNumberException("Invalid Phone Number");
      this.type = type;
      this.phone_no = phone_no;    }
```

```
    public void display() {
        System.out.println("Type: " + type);
        System.out.println("Phone Number: " + phone_no);    }}
public class kopilamobileexception {
    public static void main(String[] args) {
        try {
            Mobile m1 = new Mobile("Android", "9801234567");
            m1.display();
            Mobile m2 = new Mobile("iPhone", "98123"); // Invalid phone number
        } catch (InvalidPhoneNumberException e) {
            System.out.println(e.getMessage());        }}}
```

Output:

```
com.mycompany.kopilamobileexception.InvalidPhoneNumberExcepti
Output - Run (kopilamobileexception)
--- exec:3.1.0:exec (default-cli) @ kopilamobileexcept:
Type: Android
Phone Number: 9801234567
Invalid Phone Number
------------------------------------------------------------
BUILD SUCCESS
```

57. Create a class named Movie (id, genre). Write the object of Movie class into file named "Comedy.dat" having comedy as genre.

Definition: Serializable is a marker interface that allows an object to be converted into a byte stream (serialization) and later reconstructed (deserialization).

Syntax:

```
import java.io.Serializable;
class Movie implements Serializable {
    int id;
    String genre;
    Movie(int id, String genre) {
        this.id = id;
        this.genre = genre;    }}
```

A .dat file in Java is a binary file used to store data in raw bytes, often for serialized objects. It is not human-readable and requires streams like ObjectOutputStream and ObjectInputStream to write and read objects.

Syntax :

-To write object:

```
ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("file.dat"));
out.writeObject(object);
out.close();
```

-To read object:

```
 ObjectInputStream in = new ObjectInputStream(new FileInputStream("file.dat"));
Object obj = in.readObject();
in.close();
```

Code:

```
package com.mycompany.kopilamvieclass;
import java.io.*;
class Movie implements Serializable {
    private int id;
    private String genre;
    public Movie(int id, String genre) {
        this.id = id;
        this.genre = genre;    }
    public String getGenre() {
        return genre;    }
    public void display() {
        System.out.println("ID: " + id + ", Genre: " + genre);   }}
public class Kopilamvieclass {
    public static void main(String[] args) {
        Movie[] movies = {
            new Movie(101, "comedy"),
            new Movie(102, "action"),
            new Movie(103, "comedy")     };
        try (ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream("Comedy.dat")))
{           for (Movie m : movies) {
                if (m.getGenre().equalsIgnoreCase("comedy"))
                    out.writeObject(m);           }
            System.out.println("Comedy movies written to Comedy.dat");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());   }}
}
```

Output:

```
Output - Run (kopilamvieclass)
--- exec:3.1.0:exec (default-cli) @ kopilamviec
Comedy movies written to Comedy.dat
------------------------------------------------
BUILD SUCCESS
```

```
> Documents > NetBeansProjects > kopilamvieclass >        Search k
Comedy.dat
File   Edit   View
¬í ⊡sr #com.mycompany.kopilamvieclass.MovieòDÇMûtS¹⊡ ⊡I ⊡idL ⊡genret ⊡Ljava/lang/String;xp   et ⊡comedysq ~     gq ~ ⊡
```

58.Write a program to create a class student with data member roll and name. sort the 10 objects of this class on the basis of name.

Code:

```
package com.mycompany.kopilastdsort;
import java.util.*;
class Student {
    int roll;
    String name;
    Student(int roll, String name) {
        this.roll = roll;
        this.name = name;   }}
public class Kopilastdsort{
    public static void main(String[] args) {
        Student[] students = {
            new Student(1, "Sujan"),
            new Student(2, "Anita"),
            new Student(3, "Bikash"),
            new Student(4, "Kopila"),
            new Student(5, "Ramesh"),
            new Student(6, "Dipesh"),
            new Student(7, "Mina"),
            new Student(8, "Hari"),
            new Student(9, "Amit"),
            new Student(10, "Sita")  };
        Arrays.sort(students, Comparator.comparing(s -> s.name));
```

```
System.out.println("Sorted stds on the basis of their names");
        for (Student s : students) {
            System.out.println(s.roll + " " + s.name);    }}}
```

Output:

```
it - Run (kopilastdsort)
--- exec:3.1.0:exec (default-cli) @ kopilastdsort
Sorted stds on the basis of their names
9 Amit
2 Anita
3 Bikash
6 Dipesh
8 Hari
4 Kopila
7 Mina
5 Ramesh
10 Sita
1 Sujan
------------------------------------------------------------
BUILD SUCCESS
```

59.Create a class named Book with instance variables tile and price. Add a method named setVar to pass parameters for title and price. Add another method named showVar to display values of these variables. Now in main(), declare 4 objects of book and display the records of book that starts with "Java".

Code:

```
package com.mycompany.kopilabook;
class Book {
    String title;
    double price;
    void setVar(String title, double price) {
        this.title = title;
        this.price = price;    }
        void showVar() {
        System.out.println("Title: " + title + ", Price: " + price);    }}
public class Kopilabook {
    public static void main(String[] args) {
        Book b1 = new Book();
        Book b2 = new Book();
        Book b3 = new Book();
        Book b4 = new Book();
        b1.setVar("Java Programming", 550);
```

```
    b2.setVar("Python Basics", 400);
    b3.setVar("Java Advanced", 700);
    b4.setVar("C++ Fundamentals", 350);
        Book[] books = {b1, b2, b3, b4};
    System.out.println("Books that start with 'Java':");
    for (Book b : books) {
      if (b.title.startsWith("Java")) {  // Check if title begins with "Java"
        b.showVar();        } } }}
```

Output:

```
Output - Run (kopilabook)
--- exec:3.1.0:exec (default-cli) @ kopila
Books that start with 'Java':
Title: Java Programming, Price: 550.0
Title: Java Advanced, Price: 700.0
------------------------------------------
BUILD SUCCESS
```

60.Create a Shape interface having methods area() and perimeter(). Create two subclasses, Circle and Rectangle that implements the Shape interface. Create a class Sample with main method and demonstrate the area and perimeters of both the Shape classes. You need to handle the values of length, breadth and radius in respective classes to calculate their area and perimeter.

Theory: Interface in Java is a collection of abstract methods (methods without body). A class can implement an interface and provide the method definitions.Interfaces are mainly used to achieve abstraction and multiple inheritance.

Code:

```
package com.mycompany.kopilashapeinterface;
interface Shape {
   double area();
   double perimeter();}
class Circle implements Shape {
   double radius;
   Circle(double radius) {
      this.radius = radius;    }
   public double area() {
      return Math.PI * radius * radius;    }
   public double perimeter() {
      return 2 * Math.PI * radius;   }}
```

```
class Rectangle implements Shape {
    double length, breadth;
    Rectangle(double length, double breadth) {
        this.length = length;
        this.breadth = breadth;    }
    public double area() {
        return length * breadth;    }
    public double perimeter() {
        return 2 * (length + breadth);    }}
public class Kopilashapeinterface {
    public static void main(String[] args) {
       Circle c = new Circle(7);
        System.out.println("Circle:");
        System.out.println("Area = " + c.area());
        System.out.println("Perimeter = " + c.perimeter());
        Rectangle r = new Rectangle(10, 5);
        System.out.println("\nRectangle:");
        System.out.println("Area = " + r.area());
        System.out.println("Perimeter = " + r.perimeter());  }}
```

Output:

```
Output - Run (kopilashapeinterface)
--- exec:3.1.0:exec (default-cli) @ kopilashapeinte
Circle:
Area = 153.93804002589985
Perimeter = 43.982297150257104

Rectangle:
Area = 50.0
Perimeter = 30.0
------------------------------------------------------
BUILD SUCCESS
```

61. Write a Java program to show use of Wrapper classes.

Definition: A **Wrapper class** in Java is an object class that wraps (converts) a primitive data type (like int, char, double) into an object.Example: int → Integer, double → Double.
They are useful in collections, type conversion, and object-based operations.

Syntax:

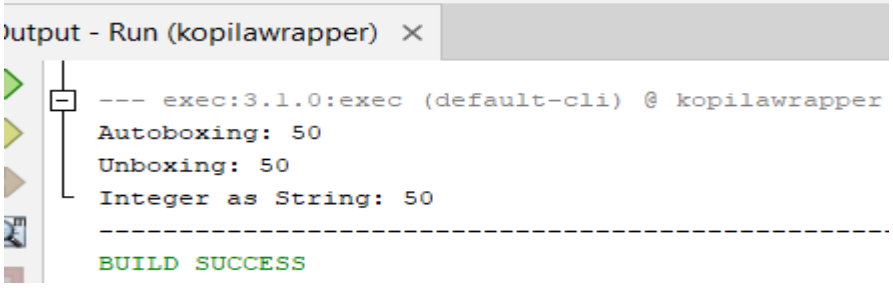
```
// Autoboxing: converting primitive to object
Integer obj = Integer.valueOf(10);
```

```
// Unboxing: converting object to primitive
int num = obj.intValue();
```

Code:

```
package com.mycompany.kopilawrapper;
class Kopilawrapper {
    public static void main(String[] args) {
        int a = 50;
        Integer obj = Integer.valueOf(a);
        System.out.println("Autoboxing: " + obj);
        int b = obj.intValue();
        System.out.println("Unboxing: " + b);
        String str = obj.toString();
        System.out.println("Integer as String: " + str);  }}
```

Output:



62.Write a Java program to illustrate inner class.

Theory : An **inner class** in Java is a class defined **inside another class**.It helps **logically group classes** that are used only in one place.Inner classes have access to **all members (even private)** of the outer class.

Syntax:

```
class Outer {
    // outer class members
    class Inner {
        // inner class members    }}
```

Code:

```
package com.mycompany.kopilainnerclass;
public class Kopilainnerclass {
    int data = 50;
    class Inner {
```

```
        void msg() {
            System.out.println("Data is " + data);   }}
    public static void main(String[] args) {
        Kopilainnerclass outer = new Kopilainnerclass();
        Kopilainnerclass.Inner inner = outer.new Inner();
        inner.msg();}}
```

Output:

```
- Run (kopilainnerclass)  ×

--- exec:3.1.0:exec (default-cli) @ kopilainnerclass
Data is 50
------------------------------------------------------------
BUILD SUCCESS
```

63.Write a Java program that shows the use of generic class.

Theory: A **generic class** allows you to create a class that can work with any data type. Type parameter <T> is used to define the type when creating an object.

Code:

```
package com.mycompany.kopilagenericclass;
class Box<T> {
    private T content;
    Box(T content) {
        this.content = content;    }
    T getContent() {
        return content;    }
    void setContent(T content) {
        this.content = content;    }}
public class Kopilagenericclass {
    public static void main(String[] args) {
        Box<Integer> intBox = new Box<>(123);
        Box<String> strBox = new Box<>("Hello");
        System.out.println("Integer Box contains: " + intBox.getContent());
        System.out.println("String Box contains: " + strBox.getContent());    }}
```

```
com.mycompany.kopilagenericclass.Kopilagenericclass
:put - Run (kopilagenericclass)  ×
--- exec:3.1.0:exec (default-cli) @ kopilag
Integer Box contains: 123
String Box contains: Hello
------------------------------------------------------------
BUILD SUCCESS
```

64.Write a Java program to show generic method.

A **generic method** in Java is a method that can operate on objects of various types while providing compile-time type safety. It allows you to write a single method that works with different data types without casting.

Syntax :

< T > returnType methodName(T parameter){ ... }

Code:

```
package com.mycompany.kopilagenericmethod;

public class Kopilagenericmethod {

   public static <T> void show(T value) {

      System.out.println("Value: " + value);    }

   public static void main(String[] args) {

      show(10);        // Integer

      show("Hello");   // String

      show(3.14);      // Double   }}
```

Output:

```
ut - Run (kopilagenericmethod)
--- exec:3.1.0:exec (default-cli) @ kopilagenericmethod
Value: 10
Value: Hello
Value: 3.14
------------------------------------------------------------
BUILD SUCCESS
```

65. Create a class Student with private member variables name and percentage. Write methods to set, display and return values of private variables in the Student class. Create 10 different objects of the student class, set the values, ad display name of Student who have highest average_marks in the main method of another class named StudentDemo.

Code:

```
package com.mycompany.kopilastudentclass;

class Student {

   private String name;

   private double percentage;

   public void setStudent(String name, double percentage) {

      this.name = name;

      this.percentage = percentage;   }

   public String getName() {

      return name;    }
```

```
    public double getPercentage() {
        return percentage;    }
    public void display() {
        System.out.println("Name: " + name + ", Percentage: " + percentage);    }}
public class Kopilastudentclass {
    public static void main(String[] args) {
        Student[] students = new Student[10];
        students[0] = new Student(); students[0].setStudent("Alice", 85.5);
        students[1] = new Student(); students[1].setStudent("Bob", 78.0);
        students[2] = new Student(); students[2].setStudent("Charlie", 92.5);
        students[3] = new Student(); students[3].setStudent("David", 88.0);
        students[4] = new Student(); students[4].setStudent("Eva", 95.0);
        students[5] = new Student(); students[5].setStudent("Frank", 82.0);
        students[6] = new Student(); students[6].setStudent("Grace", 90.0);
        students[7] = new Student(); students[7].setStudent("Hannah", 86.5);
        students[8] = new Student(); students[8].setStudent("Ian", 91.0);
        students[9] = new Student(); students[9].setStudent("Jack", 89.5);
        double maxPercentage = students[0].getPercentage();
        String topStudent = students[0].getName();
        for (int i = 1; i < students.length; i++) {
            if (students[i].getPercentage() > maxPercentage) {
                maxPercentage = students[i].getPercentage();
                topStudent = students[i].getName();    }}
        System.out.println("Student with highest percentage: " + topStudent + " (" + maxPercentage +
"%)");    }}
```

Output:

```
out - Run (kopilastudentclass)
--- exec:3.1.0:exec (default-cli) @ kopilastudent
Student with highest percentage: Eva (95.0%)
------------------------------------------------
BUILD SUCCESS
```

66.Write a program to illustrate the concept of ArrayIndexOutOfBoundsException.

Introduction: ArrayIndexOutOfBoundsException is a runtime exception in Java that occurs when a program tries to access an array with an index that is less than 0 or greater than or equal to the array's length.

Code:

```
package com.mycompany.kopilaarrayoutof;
public class Kopilaarrayoutof {
   public static void main(String[] args) {
      int[] numbers = {10, 20, 30, 40, 50};
      try {
         System.out.println("Number at index 5: " + numbers[5]);
      } catch (ArrayIndexOutOfBoundsException e) {
         System.out.println("Exception caught: " + e);    }
      System.out.println("Program continues after exception.");  }}
```

Output:

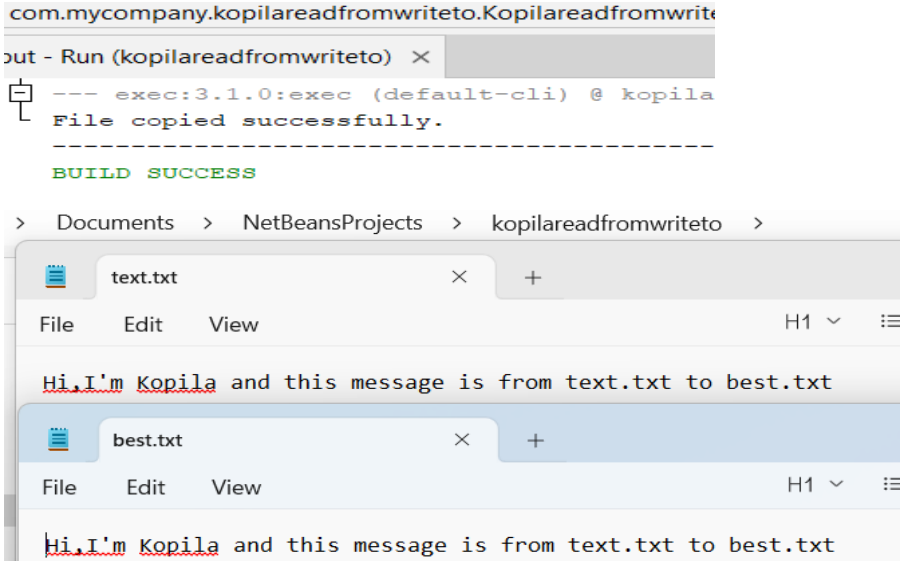
```
- Run (kopilaarrayoutof)  ×
--- exec:3.1.0:exec (default-cli) @ kopilaarrayoutof ---
Exception caught: java.lang.ArrayIndexOutOfBoundsException: Index 5 out of bounds for length 5
Program continues after exception.
------------------------------------------------------------------------
BUILD SUCCESS
```

67.Write a Java program to read data file form the file "text.txt" and write the data file into the file "best.txt".

Code:

```
package com.mycompany.kopilareadfromwriteto;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class Kopilareadfromwriteto {
   public static void main(String[] args) {
      try (FileReader fr = new FileReader("text.txt");
          FileWriter fw = new FileWriter("best.txt")) {
         int c;
         while ((c = fr.read()) != -1) {
            fw.write(c);    }
         System.out.println("File copied successfully.");
      } catch (IOException e) {
         System.out.println("An error occurred: " + e.getMessage());
 }}}
```

Output:



68.Write a Java program writes line of text to existing file. Also read the content of this file and write down on monitor.

Code:

```
package com.mycompany.kopilalinesoftextrw;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
public class Kopilalinesoftextrw{
   public static void main(String[] args) {
      String fileName = "existingfile.txt";
      try (FileWriter fw = new FileWriter(fileName, true)) { // 'true' for append mode
         fw.write("This is a new line of text.\n");
         System.out.println("Text written to file successfully.");
      } catch (IOException e) {
         System.out.println("Error writing to file: " + e.getMessage());       }
        try (FileReader fr = new FileReader(fileName)) {
         int c;
         System.out.println("File content:");
         while ((c = fr.read()) != -1) {
            System.out.print((char) c);       }
      } catch (IOException e) {
         System.out.println("Error reading from file: " + e.getMessage());     }
   }}
```

Output:

```
: - Run (kopilalinesoftextrw)
--- exec:3.1.0:exec (default-cli) @ kopilalinesoftextrw
Text written to file successfully.
File content:
This is a new line of text.
------------------------------------------------------------
BUILD SUCCESS
```

```
Documents > NetBeansProjects > kopilalinesoftextrw >
existingfile.txt
File  Edit  View
This is a new line of text.
```

69. Write a Java program reads N names of students and then sort them in ascending order.

Code:

```
package com.mycompany.kopilareadnstds;
import java.util.Scanner;
import java.util.Arrays;
public class Kopilareadnstds {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of students: ");
        int n = sc.nextInt();
        sc.nextLine();
        String[] students = new String[n];
        for (int i = 0; i < n; i++) {
            System.out.print("Enter name of student " + (i + 1) + ": ");
            students[i] = sc.nextLine();    }
        Arrays.sort(students);
        System.out.println("\nStudents in ascending order:");
        for (String name : students) {
            System.out.println(name);
        }
    }
}
```

Output:

```
- Run (kopilareadnstds) ×

--- exec:3.1.0:exec (default-cli) @ kopilareadnstds
Enter the number of students: 5
Enter name of student 1: sushila
Enter name of student 2: kopila
Enter name of student 3: aman
Enter name of student 4: kamal
Enter name of student 5: pratik

Students in ascending order:
aman
kamal
kopila
pratik
sushila
------------------------------------------------------
BUILD SUCCESS
```

70.Write a Simple GUI program that displays "hello World" in a text field. The program should display if user clicks a button.

Theory :

1. **JFrame** – A top-level container that represents the main window of a GUI application.
2. **JButton** – A GUI component that triggers an action when clicked by the user.
3. **JTextField** – A single-line text component used to display or enter text.
4. **ActionListener** – An interface used to handle action events, like button clicks.
5. **setBounds(x, y, width, height)** – Sets the position and size of a component in the container.
6. **setText(String)** – Sets the text of a JTextField or other text component.
7. **setVisible(boolean)** – Makes the GUI visible or hidden.
8. **setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)** – Ensures the program exits when the window is closed.
9. **Lambda Expression** – A compact way to represent an implementation of a functional interface (like ActionListener) using ->.
10. **Functional Interface** – An interface with a single abstract method, e.g., ActionListener.
11. **ActionEvent** – An event triggered when a user performs an action like clicking a button.

Code:

```
package com.mycompany.kopilaguihelloworld;

import javax.swing.*;
```

```
import java.awt.event.*;

public class Kopilaguihelloworld {

    public static void main(String[] args) {

        JFrame f = new JFrame("Kopila's GUI");

        JTextField t = new JTextField();

        t.setBounds(50,50,150,30);

        JButton b = new JButton("Click");

        b.setBounds(50,100,100,30);

        b.addActionListener((ActionEvent e) -> {

            t.setText("Hello World");    });

        f.add(t); f.add(b);

        f.setSize(300,300);

        f.setLayout(null);

        f.setVisible(true);

        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);    }}
```

Output:

71. Write GUI program using Swing components to find sum and difference of two numbers. Use two text fields for giving input and a label for output. The program should display sum if user presses mouse and difference if user release mouse.

Theory:

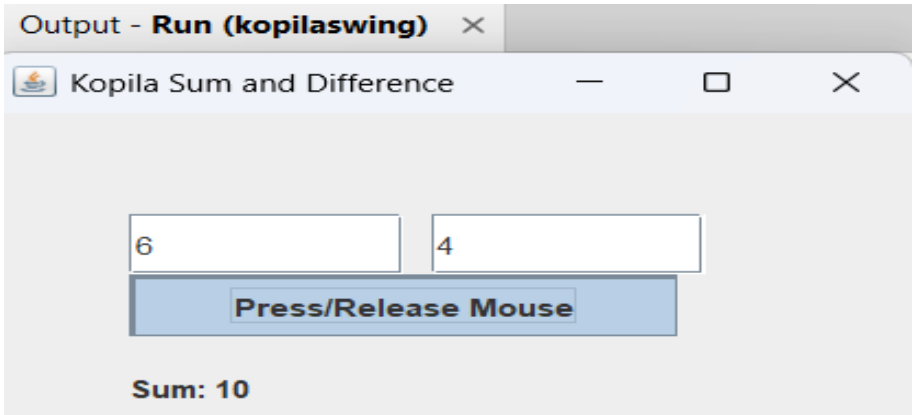
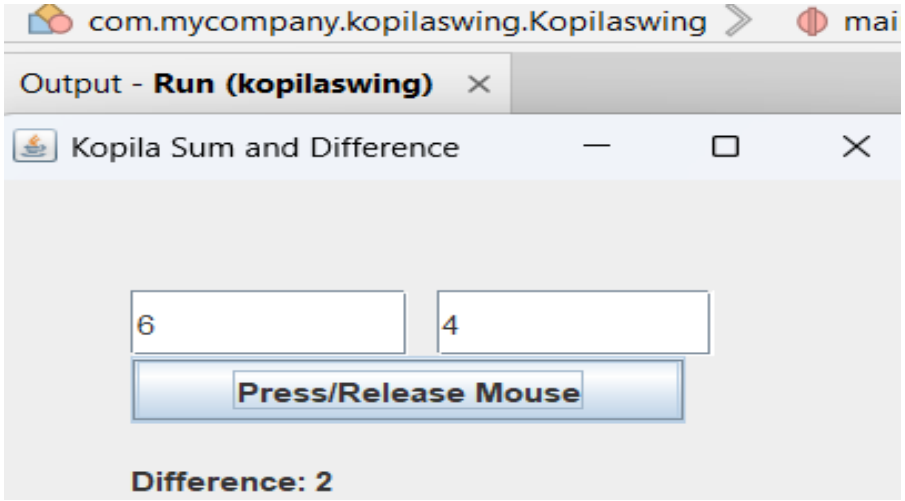
JLabel – To display output (sum/difference).

MouseListener – Handles mouse events like mousePressed and mouseReleased.

Lambda Expression – Can be used for functional interfaces, but MouseListener has multiple methods, so we implement it using an anonymous inner class here.

Code:

```
package com.mycompany.kopilaswing;

import javax.swing.*;

import java.awt.event.*;

public class Kopilaswing {

    public static void main(String[] args) {

        JFrame frame = new JFrame("Kopila Sum and Difference");

        JTextField num1 = new JTextField();

        JTextField num2 = new JTextField();

        JLabel resultLabel = new JLabel("Result: ");

        num1.setBounds(50, 50, 100, 30);

        num2.setBounds(160, 50, 100, 30);

        resultLabel.setBounds(50, 120, 250, 30);

        JButton button = new JButton("Press/Release Mouse");

        button.setBounds(50, 80, 200, 30);

        button.addMouseListener(new MouseAdapter() {

            @Override

            public void mousePressed(MouseEvent e) {

                try {

                    int a = Integer.parseInt(num1.getText());

                    int b = Integer.parseInt(num2.getText());

                    resultLabel.setText("Sum: " + (a + b));

                } catch (NumberFormatException ex) {

                    resultLabel.setText("Enter valid numbers!");      }}

            @Override

            public void mouseReleased(MouseEvent e) {

                try {

                    int a = Integer.parseInt(num1.getText());

                    int b = Integer.parseInt(num2.getText());

                    resultLabel.setText("Difference: " + (a - b));

                } catch (NumberFormatException ex) {

                    resultLabel.setText("Enter valid numbers!");   } }});
```

```
frame.add(num1);
frame.add(num2);
frame.add(button);
frame.add(resultLabel);
frame.setSize(350, 200);
frame.setLayout(null);
frame.setVisible(true);
frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);    }}
```

Output: